

MemAura: 面向智能环境中个性化大语言模型推理的结构化上下文记忆

SIYUAN LIU, 香港科技大学 (广州), 中国 HUANGXUN CHEN*, 香港科技大学 (广州), 中国

基于历史上下文提供智能环境中的个性化服务已成为一项重要需求。一个前景广阔的方向是利用大语言模型强大的推理能力。然而, 实现个性化需要持久的上下文记忆。关键挑战在于如何高效组织长期的设备和传感器日志, 以及如何从中提取用户行为模式和偏好。为应对这一挑战, 我们提出了MemAura, 一个面向智能环境的记忆管理系统。MemAura包含三个核心模块: (1) 记忆图管理器, 它高效组织上下文记忆, 支持快速检索, 并向大语言模型提供用户模式和偏好信息; (2) 周期性模式预测器, 它在给定时间戳预测周期性的行为模式和偏好; (3) 用户画像方案, 它将指令转化为明确的意图单元, 并随时间推移增量更新和遗忘高层级用户画像。

我们在两个指令数据集上, 将 MemAura 与多个基线进行了评估。以 GPT-4o 为骨干模型时, 它实现了 100% 的准确率和 38%–43% 的个性化率。当部署轻量级本地模型时, MemAura 仍能达到高达 95% 的准确率, 平均个性化率最高可达 37.5%。然而, 当使用相同的轻量级大语言模型时, Vanilla 方法仅能达到 28%–56% 的准确率, 个性化率最高仅为 21%。向量检索增强生成方法在轻量级模型上实现了 54–83% 的准确率和最高 25% 的个性化率。虽然它优于 Vanilla 方法, 但仍落后于 MemAura。结果表明, MemAura 在不同规模的大语言模型上均能保持高准确率高个性化率, 且具有实用的令牌使用量和延迟。我们还进行了一系列实验来验证 MemAura 的鲁棒性。一项包含 8 名参与者的用户研究进一步证实了其可用性。参与者全面体验并评估了其卓越的性能和服务质量。我们的工作使大语言模型能够更好地理解用户意图, 并从上下文记忆中高效提取模式和偏好, 从而使智能生活空间更加高效、具备情境感知能力并以用户为中心。

CCS 概念 概念: • 以人为中心的计算 → 普适与移动计算系统和工具。

其他关键词和短语: 大语言模型, 个性化记忆管理

ACM 参考文献格式:

Siyuan Liu 和 Huangxun Chen. 2026. MemAura: 面向智能环境中个性化大语言模型推理的结构化上下文记忆。《ACM交互式、移动、可穿戴和普适技术汇刊》10, 2, 第55篇文章 (2026年6月), 43 页。 <https://doi.org/10.1145/3810196>

1 引言

实现一个真正智能的生活环境一直是人们追求的理想, 智能助手 [12, 13, 50] 如Alexa和Siri代表了实现这一目标的早期努力。它们能有效处理显式指令, 例如, “将客厅空调温度降低两度”。然而, 它们常常

*通讯作者。

作者地址: Siyuan Liu, 香港科技大学 (广州), 中国广东广州, sliu268@connect.hkust-gz.edu.cn; Huangxun Chen (通讯作者), 香港科技大学 (广州), 中国广东广州, huangxunchen@hkust-gz.edu.cn。



本作品采用知识共享署名-非商业性使用-禁止演绎 4.0 国际许可协议进行许可。© 2026 版权所有/作者所有。

ACM 2474-9567/2026/6-ART55 <https://doi.org/10.1145/3810196>

《ACM交互式、移动、可穿戴和普适技术汇刊》, 第10卷第2期, 文章55。发表日期: 2026年6月。

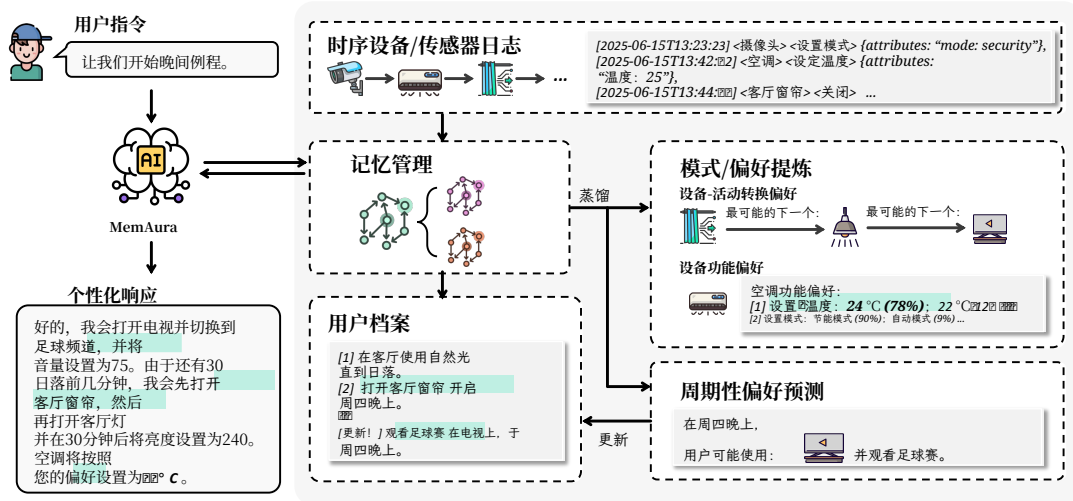


图 1. 使用场景概览。MemAura 从原始日志中构建其记忆，并能够蒸馏和预测行为模式与偏好，以响应用户指令。每次交互后，MemAura 会根据响应结果进一步更新用户档案，使系统能够适应用户习惯的演变。

难以处理模糊或复合的请求，例如“让我们开始晚间例程吧”。这类指令既未指定目标设备，也未说明预期动作，且没有单一明确的响应，使得系统难以准确推断用户意图。如果由人类管家来处理类似的请求，他会基于对雇主偏好和日常习惯的理解来推断其具体需求。例如，知道“今天是星期四，雇主通常在晚上观看足球赛并喝啤酒”，管家会迅速识别相关设备，并根据请求的上下文语义采取相应行动。这种响应本质上是高度个性化的，得益于管家对雇主的深刻了解，但代价是雇佣一位管家的成本。我们的目标是开发一个智能系统，能够实现类似水平的上下文推理和个性化适应能力，如图1所示。

大语言模型（LLMs）的最新进展已展现出实现个性化服务的有前景的推理能力。一些研究 [31, 49] 已在智能手机等个人设备上对此进行了探索，这些设备上的用户交互历史被良好地组织和存储，使得大语言模型能够提取偏好模式并推理出个性化响应。然而，将此类个性化扩展到整个环境仍然更具挑战性且不够成熟。现有的努力 [15, 23, 28, 42] 主要侧重于基于即时环境状态来解释用户指令，而对有效管理跨多个设备的历史用户交互或系统性地提炼用户偏好以提供更量身定制的响应方面关注有限。基于我们的调查，我们确定了在实现一个能够理解用户偏好并根据用户指令提供个性化服务的智能环境时所面临的三个关键挑战。

- **CH1: 多用户多设备交互历史管理。** 为提供个性化服务，必须以一种有组织的方式记录用户行为和操作。个人智能手机上的用户交互可以从单一设备持续收集，然而，智能环境内的交互涉及多个设备，并展现出固有的时空特性。此外，此类环境通常由多位居民共享。因此，需要一个有效的方案来

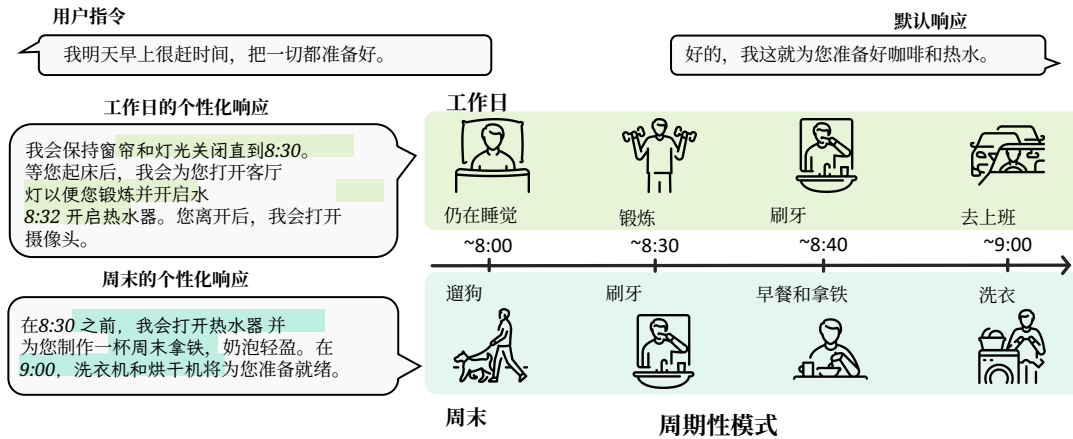


图 2. 一个代表性用例，突显了预测周期性模式对于提供个性化智能服务的重要性。通过显式预测用户在不同日子里的多样化周期性早晨惯例，系统能够消除相同但未明确说明的惯例准备请求的歧义，从而实现对相应一组惯例事件的准确响应和适当资源的主动资源调配。

管理 你好跨多个用户和设备的交互历史轨迹，使大语言模型能够高效检索
p精确一个并在需要时提供上下文相关的信息。

- **CH2: 用户特定偏好与模式提炼。**如图2所示，当系统明确知晓用户的偏好以及跨情境（例如工作日与周末）的周期性行为模式时，它便能有效消除相同但未明确说明的惯例准备请求的歧义，并提供准确的响应。具体而言，在工作日，系统预测用户通常睡到早上8:30，锻炼10分钟，然后洗漱后去上班。因此，系统可以主动为这些惯例事件预置资源。相比之下，在周末，预测的用户行为呈现出不同的模式，例如更早起床遛狗、洗漱、在家吃早餐以及洗衣服，因此准备的资源也应相应调整。原则上，在智能环境中收集的原始交互日志可以捕捉全面的用户信息。然而，若没有有效的提炼和显式预测，它们会给下游推理模型带来沉重负担，使其难以识别用户偏好并推断周期性模式，且往往仍会导致不尽人意的性能。如图3所示，我们的研究揭示了LLM规模与所提供的用户偏好上下文精度之间的权衡：直接提供原始日志（图3中的vanilla）需要一个强大的LLM（例如GPT-4o等在线API）来推断个性化响应，而提供提炼后的偏好上下文（图3中的MemAura）则能让较小的LLM实现相当的性能。因此，有效地从原始交互日志中提炼出显式的用户偏好和周期性模式至关重要，以实现可本地部署、更经济且隐私友好型的方案。

- **CH3: 用户档案管理。**每个用户都表现出多个维度的偏好，其中某些偏好可能是稳定的，反映了长期习惯，而其他偏好可能是暂时的，并且可能快速变化。为了使大语言模型能够有效地进行推理并提供个性化服务，需要一个高效的方案来管理这些偏好，并根据用户不断变化的行为和发出的指令，动态调整其优先级。

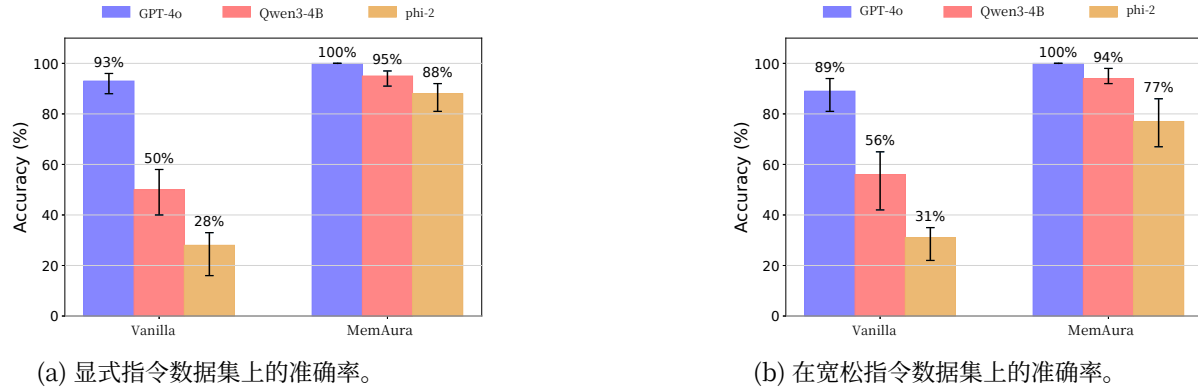


图3. Vanilla（即直接提供原始日志）与MemAura的准确率对比。当使用GPT-4o等大规模大语言模型时，Vanilla和MemAura均能持续保持高准确率。然而，当使用更轻量级的大语言模型时，Vanilla的准确率急剧下降，而MemAura得益于其结构良好且具备上下文感知能力的记忆管理，其准确率水平仍可与大规模大语言模型相媲美。

为了应对这些关键挑战，我们提出了MemAura，这是一个为智能环境中的基于大语言模型的服务提供持久化、个性化上下文记忆的系统。为了解决CH1，MemAura配备了一个**记忆图管理器**，这是一个基于图的框架，旨在高效组织多用户多设备交互历史，作为大语言模型服务的上下文输入。该图将所有原始日志构建成用户-设备时序事件记忆，使大语言模型能够快速定位、检索和提取相关的历史信息。此外，还衍生出两个子图：设备与活动转移偏好图和设备功能使用图，以更好地表示用户特定的模式，从而促进大语言模型推理。为了解决CH2，MemAura引入了**周期性模式预测器**，它从记忆图中提取用户特定的事件序列，并应用基于BERT的模型来预测用户在给定时间戳下的周期性偏好，从而为大语言模型推理提供更明确、更具上下文感知的指导。为了解决CH3，MemAura引入了**用户画像方案**，该方案由一个双层用户偏好队列、一个用户意图分解器以及一个意图理解与画像更新之间的交互机制组成。具体来说，在接收到用户指令后，系统会将其分解为一组细粒度的意图单元，用于顺序大语言模型推理。然后，系统调用周期性模式预测器来获取用户当前的上下文偏好。推理完成后，被发现有用的偏好要么被插入到用户偏好队列中，如果它们已存在于队列中，则提升其优先级。随着时间的推移，队列中的前 k 个条目自然形成一个动态用户画像，捕捉了用户的代表性偏好，从而有效地指导大语言模型的个性化推理和自适应决策。

我们的主要贡献总结如下：

(1) 我们认识到管理用户特定的交互历史对于实现智能环境的重要性，并强调了其中的关键挑战。据此，我们提出了MemAura，这是一个能够实现持久化、个性化上下文记忆以增强智能环境中基于大语言模型的服务的系统。(2) MemAura 主要具备以下特性：一个用于高效组织多用户、多设备交互序列的记忆图管理器，一个用于推断用户在给定时间戳最可能偏好的周期性模式预测器，以及一个用于构建并随时间演进用户各种偏好的用户画像方案。(3) 我们扩展了现有的生活空间（智能家居）指令数据集 [23]，纳入了需要个性化的更丰富的指令，并进一步构建了一个用于工作空间（智能办公室）的新指令数据集。每个数据集包含40条显式指令和28条宽松（未明确指定）指令。此外，我们构建了一个记忆上下文数据集，以多样化地模拟智能环境中的用户交互历史。

我们记忆上下文数据集的真实性基于一个真实世界数据集 [38]。我们利用大语言模型来丰富其多样性，然后在VirtualHome物理引擎中回放生成的轨迹 [40] 以进行验证，仅保留那些满足真实世界交互逻辑的轨迹，从而确保我们数据集的真实性。

(4) 我们进行了全面评估，以证明MemAura的有效性和可用性。利用我们的数据集，我们评估了MemAura提供的精确情境信息如何使各种规模（尤其是较小规模）的大语言模型能够在可接受的延迟和令牌成本内执行令人满意的个性化推理。以GPT-4o为骨干模型时，MemAura实现了100%的任务准确率和38%–43%的个性化率。当使用轻量级大语言模型（如Qwen3-4B和Phi-2）时，MemAura实现了高达95%的准确率，平均个性化率高达39%。然而，原始方法（直接将原始日志输入模型）尽管在以GPT-4o为骨干模型时取得了相当不错的性能，但在使用轻量级大语言模型时，仅获得28%–56%的准确率和最多21%的个性化率。对于另一个基线向量检索增强生成 [25]，（使用GPT-4o），其准确率与原始方法相当，个性化率最高达30%；而对于轻量级模型，其准确率为54–93%，个性化率为15–25%。虽然向量检索增强生成优于原始基线，但仍明显不及MemAura。此外，我们设计了一系列压力测试来模拟现实世界的约束条件，包括记忆长度较短的冷启动场景、设备中断以及日志噪声（错误的日志条目）。结果表明，MemAura在这些条件下依然保持稳健。此外，我们的消融研究验证了每个独立设计组件的有效性，我们还招募了一组参与者来评估个性化质量和整体用户体验。

2 MEMAURA 设计

2.1 概述与工作流程

图 4 展示了 MemAura 的概览。当接收到一条用户指令时，MemAura 首先使用意图分解器 (§ 2.4.2) 将指令分割成细粒度的意图单元，然后对这些单元进行顺序处理。对于每个推理和服务步骤，MemAura 首先查询记忆图管理器 (§ 2.2) 以检索上下文信息，主要是通过识别高度相关的子图作为相关上下文。此外，系统会调用周期性模式预测器 (§ 2.3) 来获取针对当前时刻的用户特定偏好预测。最后，大语言模型还会结合一个从过往交互中蒸馏得出的高层级用户画像 (§ 2.4.1)，为每个意图单元生成自适应的个性化响应。这些响应随后被整合以产生最终的综合性输出。推理过程结束后，一个事后处理程序 (§ 2.4.3) 会更新双层用户偏好队列。被识别为有意义的偏好要么被插入队列，如果已存在则提升其位置，而过时的条目可能会随着用户习惯的演变而被移除。因此，队列中领先的前 k 个条目会随着时间动态演变，为后续的推理和个性化提供有效的参考。

2.2 记忆图管理器

智能环境中的日常交互日志记录了每位居民的偏好与习惯。与基于对话的用户交互数据不同，与智能设备的交互天然地呈现出由用户的物理情境和活动模式所塑造的时空特性。受此观察启发，我们设计了全局记忆图，如图 5 所示，它是一种富有表现力且高效的结构，用于组织原始设备日志，以实现快速检索和情境访问。我们进一步推导出两个子图，如图 6 所示，以显式地表征用户行为模式，从而实现更高效的查询与利用。记忆图管理器为大语言模型提供了便捷的接口，用于检索精确情境信息，从而促进个性化响应的生成。

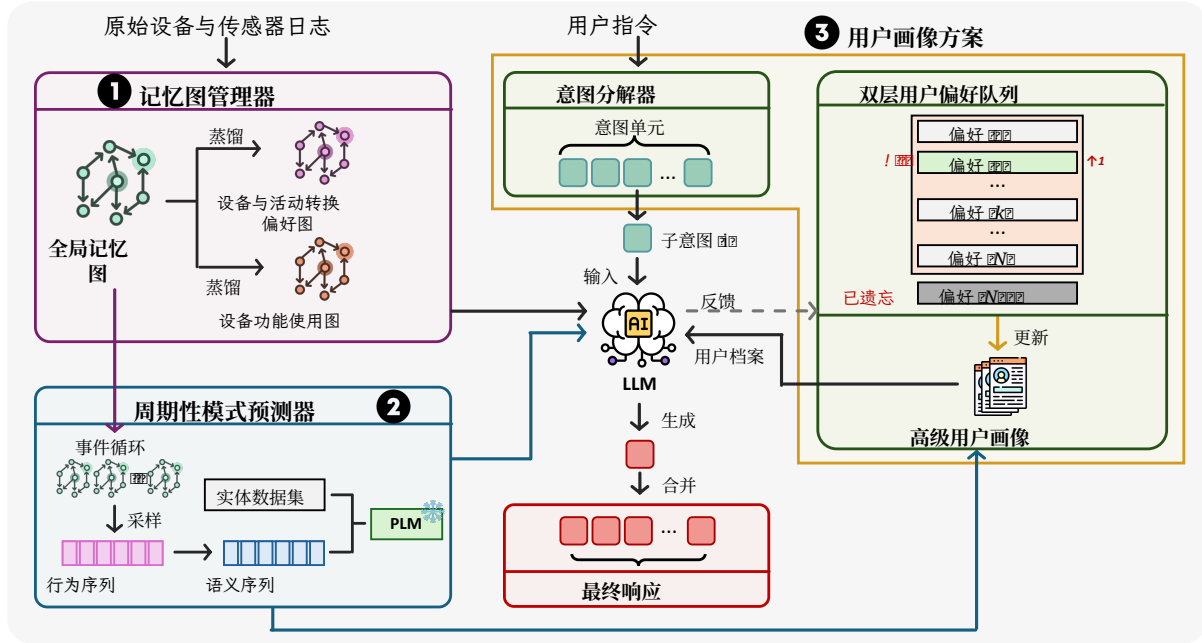


图4. MemAura系统概览。该系统由三个核心模块组成：(1) 记忆图管理器，它高效组织上下文记忆，支持快速检索，并向大语言模型提供用户行为模式和偏好信息；(2) 周期性模式预测器，它在给定时间戳预测周期性的行为模式和偏好；(3) 用户画像构建方案，它由一个双层用户偏好队列和一个用户意图分解器组成。该方案会随时间推移增量更新并遗忘高层级用户画像。

2.2.1 全局记忆图构建。我们提出了一种有向、带标签的异构图 G ，它由智能设备和传感器收集的日志构建而成。它包含八个顶点类别：

$$V = \bigcup_X V_X, X \in \{U, D, F, AC, E, I, AR\}$$

这些顶点类别分别表示用户（例如，爱丽丝）、设备（例如，咖啡机）、传感器（例如，厨房_入口_传感器）、功能（例如，打开_）、活动（例如，烹饪）、事件（例如，爱丽丝烹饪，爱丽丝打开咖啡机）、意图（例如，在卧室准备睡觉）和区域（例如，厨房）。因此，我们定义了节点属性函数 $\ell_V(V_X) = X$ 。对于事件，以下属性函数分别将每个事件映射到其在 $\mathbb{T}$ 中的时间戳和其在 $\mathbb{N}$ 中的序列号：

$$\tau : V_E \rightarrow \mathbb{T}, \sigma : V_E \rightarrow \mathbb{N}$$

图边 E 包含九个类别及相应的标签 ℓ_E ，如表1所示。由此生成的图 $G = (V, \ell_V, \tau, \sigma, E, \ell_E)$ 统一了静态实体（如设备、设备功能、居民和日常活动）以及动态事件节点扩展，并包含了它们之间的相互作用。

图5展示了全局图的部分视图 G ，其中显示了几个初始事件节点作为示例。每个事件节点记录一个交互事件。例如，事件1在图中代表事件“爱丽丝打开厨房灯。”每个事件节点形成一个唯一的有向环，其结构为事件 $\rightarrow$ 设备 $\rightarrow$ 功能 $\rightarrow$ 用户 或 事件 $\rightarrow$ 活动 $\rightarrow$ 用户，该环记录了与该事件相关的每个参与者和细节。例如，事件1

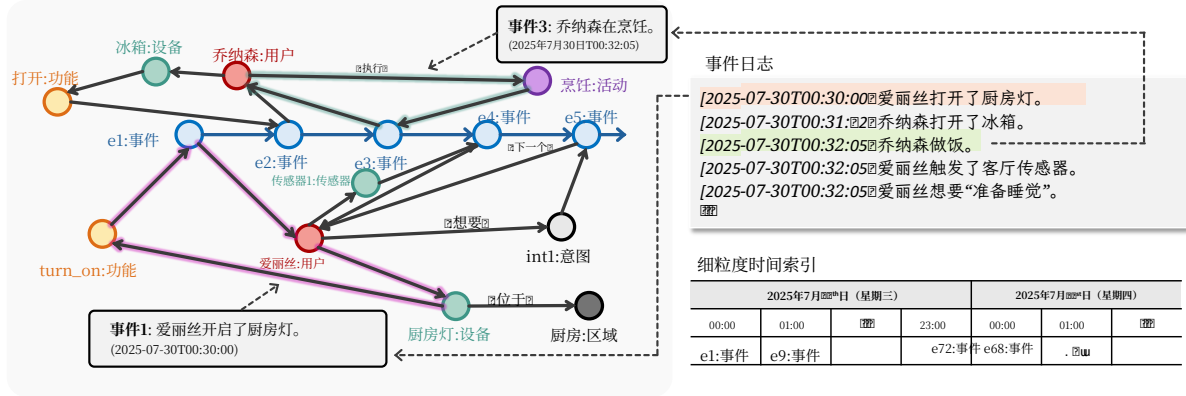


图5. 全局记忆图结构示例。全局图存储了多位居民共享的长期交互日志。每个事件节点代表由设备或传感器记录的一个动作，并形成唯一的有向环，从而捕捉到该事件涉及的所有参与者。系统采用时间索引来实现对特定事件的快速检索。

表 1. 全局记忆图中的边类型。

符号	具体说明	标签
E_{OP}	用户 $\rightarrow$ 设备	操作
E_{NX}	用户 $\rightarrow$ 传感器	触发
E_{LO}	(设备 $\cup$ 传感器) $\rightarrow$ 区域	位于
E_{EX}	设备 $\rightarrow$ 功能	执行
E_{DO}	用户 $\rightarrow$ 活动	DO
E_{DO}	用户 $\rightarrow$ 意图	想要
E_{GN}	(功能 $\cup$ 活动 $\cup$ 传感器) $\rightarrow$ 事件	生成
E_{PB}	事件 $\rightarrow$ 用户	已执行
E_{NX}	事件 $\rightarrow$ 事件	BY NEXT

对应于唯一的有向环 e_1 : 事件 $\rightarrow$ 厨房_灯: 设备 $\rightarrow$ 打开_: 功能 $\rightarrow$ 爱丽丝: 用户 $\rightarrow e_1$: 事件。通过用适当的介词连接这些节点，该事件可以表达为自然语言描述“爱丽丝打开厨房灯。”由于智能环境中的设备和传感器可能分布在不同的物理空间，该图 G 还包含一个名为 区域 的超节点。每个设备或传感器节点通过一条边 E_{LO} 链接到其对应的区域，以区分它们的空间分布。简而言之，我们的图设计对大语言模型友好。所有节点和边都具有语义化的名称，并且事件周期可以转换为包含完整上下文信息的自然语言描述，使大语言模型能够高效访问和解释底层数据。

2.2.2 细粒度时间索引。在图 G 之上，我们引入了一个两级索引 $I: D \times H \rightarrow V_E \cup \perp$ ，以支持高效的每日/每小时事件检索。具体来说，

$$I(d, h) = \arg \min_{v_e} \{ \tau(e) \mid \tau(e) \in [T_{d,h}, T_{d,h+1}) \}$$

$$= \perp, \quad \text{if no such event } e \text{ exists}$$

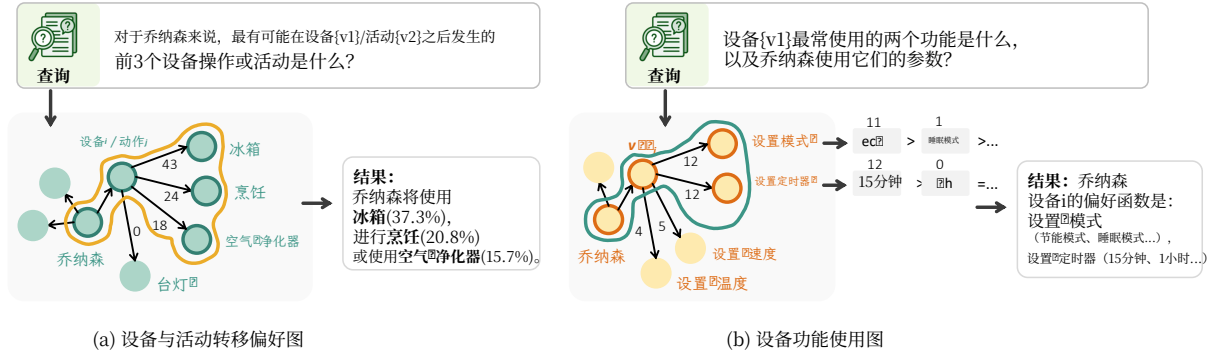


图6. 两个衍生子图结构的示例。这两个子图由全局图生成。左侧是设备与活动转移偏好图，基于统计转移概率识别最可能的下一个设备操作或活动。右侧的子图，即设备功能使用图，有效捕捉了用户与设备*i*交互时最常使用的功能和参数配置。

它将给定的日期 d 和小时 h 映射到该持续时间内的第一个事件，其中 $T_{d,h}$ 表示日期 d 上小时 h 的起始时间戳。

2.2.3 设备与活动转移偏好图。 每个用户的事件档案可以表示为一个设备操作、传感器日志和活动执行的序列。其中，设备操作和活动执行包含了丰富的上下文细节，封装了复杂的行为模式和用户偏好。虽然全局图 G 是全面的，但有时可能过于详细，难以直接推断特定用户在不同操作和活动之间的转移概率。为了让大语言模型能高效地获取此类关于用户行为模式的统计洞察，我们从全局图 G 中衍生出一个**设备-活动转移偏好图** G_T^u 。

如图6(a)所示，此图主要包括设备顶点 V_D 和活动顶点 V_{AC} 。我们检查每条边 $\{e_m, e_n\} \in E_{NX}$ ，并识别与端点相连的关联设备或活动节点。定义如下：

$$\pi(e_m) = v_i, \pi(e_n) = v_j$$

其中 $v_i, v_j \in V_D \cup V_{AC}$ 。然后我们建立一条有向边 $(v_i, v_j) \in E_T^u$ ，并将关联的边属性 w 增加1以反映转移频率。生成的图 $G_T^u = (V_D \cup V_C, E_T^u, w_T^u)$ 总结了居民的历史活动转移模式，使大语言模型能更轻松地提取有意义的信息。

图6展示了图 $G_T^u = (V_D \cup V_C, E_T^u, w_T^u)$ 及其用法的一个示例，具体而言，系统如何高效地回答大语言模型查询，例如“对于乔纳森，在设备*i*和活动*j*之后最可能发生的Top 3设备操作或活动是什么？”我们首先识别所有连接到乔纳森节点的设备*i*或活动*j*节点，然后定位每个相邻事件中涉及的对设备或活动节点。统计出现次数并进行排序，以获得最频繁的转移。如本例所示，在使用设备*i*之后，乔纳森最常与冰箱交互（43次）、开始烹饪（24次）或使用空气净化器（18次）。此类用户偏好可被解释为个性化概率分布。因此，具有最高转移概率的节点代表了乔纳森在操作设备*i*之后后续动作的Top-K偏好。

2.2.4 设备功能使用图。 不同用户可能对同一设备的各种设备功能表现出不同的偏好。为了让这些模式在大语言模型推理中更加显著，我们提取了一个用户特定的

设备使用加权图 G_F^u 从全局图中 G , 如图 6(b) 所示。此图主要由设备顶点 V_D 和功能顶点 V_F 组成。我们合并所有具有相同端点的边 $\{v_d, v_f\} \in E_{EX}$, 并将边属性 w 指定为此类边的总数, 这反映了用户 u 在设备 d 上调用功能 f 的次数。每个功能节点元数据中的归属字段在每次使用后也会被加权和计数, 从而使系统能够记录更细粒度的偏好设置。

例如, 如图6所示, 当大语言模型查询乔纳森对设备 i 的偏好功能和参数时, 可以直接从图 G_F^u 中获取此信息。具体来说, 首先根据权重对所有连接设备 i 与其关联功能节点的边进行排序。然后, 每个功能节点内的参数按其自身的加权频率排序。最终得到的排序结果代表了乔纳森对设备 i 的Top-K偏好设置。在此示例中, 偏好配置是将温度设置为24°C并调整为节能模式。

2.3 周期性模式预测器

除了记录在上述图结构中的历史数据外, 为了帮助大语言模型生成个性化响应, 了解用户在发出指令时最可能做什么是非常有益的。此预测任务可以通过多种方式处理, 例如, 将所有上下文信息输入大语言模型。然而, 考虑到推理成本、延迟和数据隐私, 我们将其表述为**掩码标记预测任务**, 并训练一个轻量级的周期性模式预测器来高效执行预测。我们的设计概览如图7所示。

2.3.1 预测器训练。 我们的基本思路是采用BERT [7] 作为预测器主干, 利用其强大的捕获语义依赖的能力。然后, 我们从全局图 G 中提取时间-语义信息, 构建一个领域特定数据集, 用于对基础BERT模型进行微调。

具体来说, 如前所述, 全局图中的每个事件都对应一个特定的时间戳, 并封装了一套完整的实体和关系。因此, 我们按时间将全局图分割成一个历史子图序列 $G_{t_1:t_m}$, 每个子图代表从 t_1 到 t_m 的连续时间窗口内发生的事件。从每个子图中, 我们识别出需要预测的实体 (设备、活动和意图), 并将它们编译成一个实体列表。接着, 我们对事件周期进行基于频率的采样, 为列表中更频繁出现的实体分配更高的采样权重。这个过程产生一个时间有序序列, 其中每个事件表示为一个四元组 ($user, relation, entity, timestamp$)。然后, 我们将每个四元组转化为一个语义连贯的句子 s_i , 其格式为:

$$s_i = time, user relation entity$$

多个句子 s_i 随后按时间顺序连接起来, 并分别在开头和结尾添加特殊标记 $[CLS]$ 和 $[SEP]$, 以形成一个用于BERT的完整样本 S_q :

$$S_q = [CLS] s_1, s_2, ..., s_i. [SEP]$$

我们应用一个滑动时间窗口来生成多个此类样本 S_q , 从而形成一个领域特定数据集, 用于微调基础BERT模型以获得我们的周期性模式预测器。

2.3.2 预测器推理过程。 如图7右侧所示, 当已知一系列历史事件且目标是预测乔纳森在特定时间会做什么时, 这可以表述为一个查询 q 。我们将此类查询定义为以下三种类型:

$$\begin{aligned} q_w(time, user) &= time, user \text{ wants ?} \\ q_o(time, user) &= time, user \text{ operates ?} \\ q_d(time, user) &= time, user \text{ does ?} \end{aligned}$$

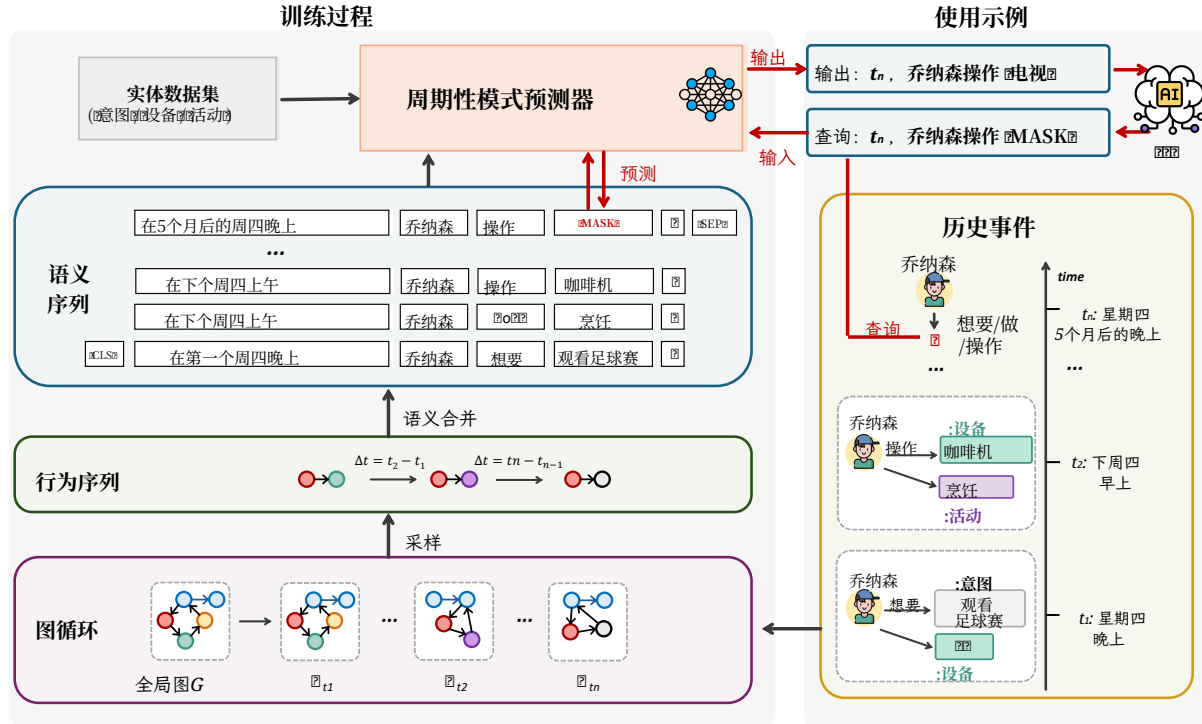


图7. 周期性模式预测器概述。我们对全局图中的行为周期执行基于频率的采样，并计算采样事件之间的时间间隔以构建行为序列。这些序列随后被组织成语义连贯的句子，形成语义序列。用户在特定时间点的潜在动作被视为语义序列中待预测的掩码实体。我们提取所有可能的实体，并通过BERT将它们与语义序列一起进行微调。最终，周期性模式预测器可以进行预测。

训练好的预测器可以处理一个查询 q 以做出预测，记作 $P_t^{S, q}$ ：

$$P_t^{S, q} = \text{Predictor}(S_q)$$

预测输出被视为用户特定周期性模式，表明用户在给定时间可能做什么。与调用大语言模型进行此类推理相比，我们的预测器紧凑、高效，并且同样实现了高准确率。此外，由于整个过程可以在本地服务器上负担得起，它降低了使用云服务带来的隐私泄露风险。

2.4 用户画像方案

2.4.1 双层用户偏好队列。从历史行为记录中，可以推断出特定用户的多种周期性模式。然而，人类行为和偏好常常动态演变。例如，全局图中的近期历史可能显示乔纳森通常在周四晚上观看足球，同时也喜欢自然光和新鲜空气。因此，需要一个有效的结构来捕捉用户偏好的各个方面，管理其优先级和时效性，并随时间动态更新。因此，如图8所示，我们设计了一个由多个记录用户偏好的条目组成的双层用户偏好队列。同时，

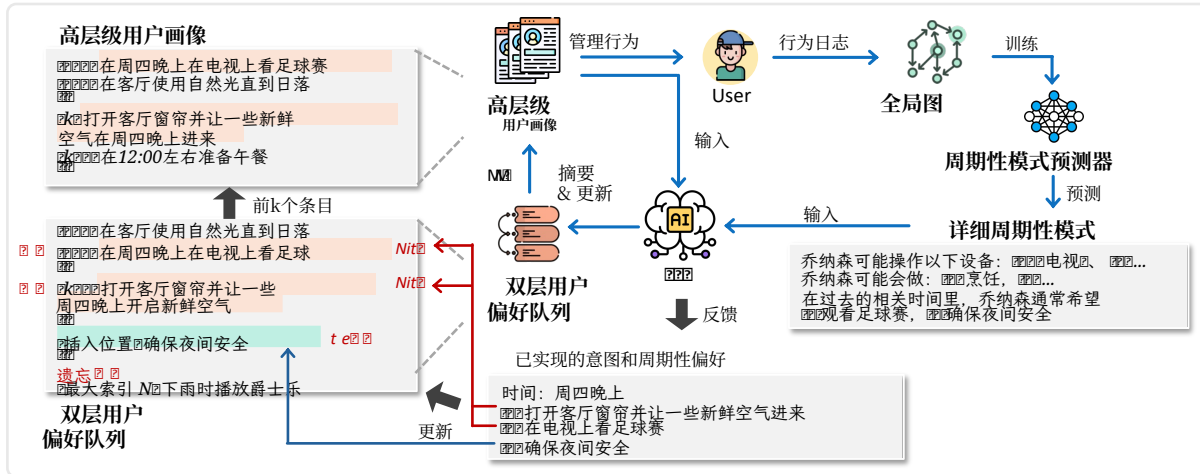


图 8. 双层用户偏好队列概览。用户的偏好会导致某些行为频繁发生，这使得这些行为在全局图中得到越来越多的体现，从而更有可能被周期性模式预测器预测到。这些偏好也体现在重复的指令意图中。在大语言模型完成其推理后，它会提供反馈，表明响应中采纳了哪些偏好项。双层用户偏好队列维护着一个类似缓存的结构：每次大语言模型报告一个被使用的偏好项，该项就会被添加到队列中。如果该项已存在，则其优先级会提高。队列有固定大小，因此长时间未被访问的项会被移除，即被遗忘。更新后剩余的前 k 项构成下一轮推理的有效高层级用户画像。更新过程是由大语言模型在系统空闲时间执行的后台任务。

排名前 k 的条目被视为高层级用户画像，用以表征用户的代表性偏好，这些偏好指导大语言模型的个性化推理和自适应响应生成。

2.4.2 用户意图分解器。为了准确更新用户偏好队列，我们认为在系统运行时进行意图理解是一个关键组成部分。为实现精确的意图解释，我们设计了一个意图分解器模块，专门用于处理诸如“让我们开始晚间例程”这类先前提到的模糊指令。该模块将用户指令分解为显式意图单元。我们采用了一种精心设计的少样本提示策略，如图9所示，以引导大语言模型执行有效的初始意图分解。鉴于严格的输出约束，意图分解器很少在我们的评估中产生不切实际的结果。尽管如此，我们为用户提供了灵活的选项来审核和完善系统的推理。如图9所示，意图分解器会显式输出意图单元，随后用户可以选择执行所有单元或仅执行一个子集，或者请求修订特定单元。在用户确认后，负责推理的大语言模型会并行处理已批准的单元，然后将结果合并为最终输出。

2.4.3 意图理解与用户画像更新的交互机制。如图8所示，在MemAura中，用户意图推理驱动着用户档案的更新，而维护良好的档案反过来又能促进更准确、更个性化的LLM推理。

具体来说，在每次意图推理之后，系统会对该过程进行反思，并识别出那些对生成个性化响应有贡献的偏好条目，这些条目可能来自现有高层级用户画像，也可能来自经过训练的周期性模式预测器所提供的新发现的偏好。系统随后会相应地在用户偏好队列中更新它们的顺序。新识别出的偏好会被插入到队列中靠近中间的一个预定义位置（插入_位置），以保持表征的时效性并反映近期行为。

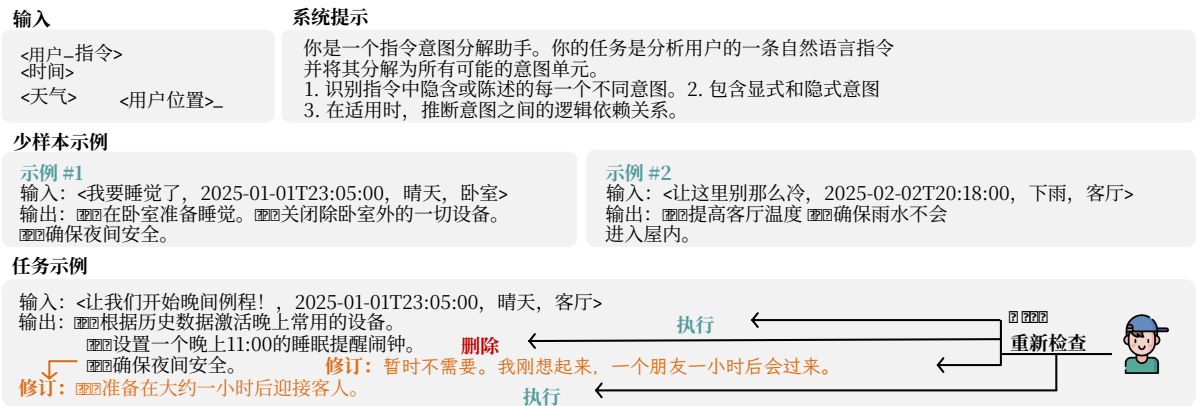


图9. 意图分解器的提示设计。在某些情况下，可能会采用基于API的大语言模型来支持此组件；因此，为最小化隐私泄露风险，输入数据保持简洁且不包含任何个人信息。在意图分解器输出意图单元后，用户可以审核它们，决定执行或丢弃哪些单元，以及通过提示大语言模型重新生成来修订哪些单元，从而在自动化与用户控制之间取得平衡。

该队列具有最大容量（最大_索引）。添加新项目会将其后的所有条目向下推，超出限制的条目将被移除，这代表了因长期不活动而被遗忘的偏好。对于现有条目，它们在队列中的优先级会提升一个位置。每次更新后的前k个条目构成下一个活跃高层配置文件，用于后续的LLM个性化推理。参数 k、插入_位置和最大_索引是可调的超参数。

这种意图理解与画像更新之间交互机制的设计原理，灵感来源于人类的短期记忆系统：只能保留有限数量的条目。那些很少被重新激活的条目会逐渐淡忘，而那些频繁被激活的条目则会随着时间的推移持续存在。在我们的场景中，如果某些行为被用户重复执行，它们会被捕获到全局记忆图中，使我们的预测器能够将其提取为偏好。同样，如果用户频繁发出特定指令，则表明其潜在的偏好，这些指令会在队列中逐渐向上提升，形成高层级用户画像的一部分。在我们的设计中，只有那些随时间持续存在并累积的偏好才被视为高层级用户画像的一部分。值得注意的是，系统将偏好更新作为后台任务执行，在推理过程中不会引入额外的延迟。

3 评估

在本节中，我们对MemAura进行详细评估。首先介绍实验中使用的指令数据集和上下文记忆数据集。然后描述评估指标、基线方法以及部署配置。最后，我们展示衡量MemAura准确率和整体性能的实验结果。我们旨在回答以下研究问题：**研究问题1**：MemAura的准确率、令牌消耗和延迟与最先进的LLM之间是否存在权衡？**研究问题2**：MemAura在使用轻量级本地模型时能否保持高准确率？**研究问题3**：当MemAura失败时，哪种类型的错误更频繁发生？**研究问题4**：MemAura是否提供显著的个性化服务？**研究问题5**：MemAura各个模块对其有效性的影响程度如何？

研究问题6: MemAura在现实约束下是否稳健, 这些约束包括设备故障、日志缺失或损坏, 以及上下文记忆有限的冷启动场景?

3.1 数据集

我们在评估中使用两种类型的数据集: 一个指令数据集用于评估系统对不同用户指令的响应, 以及一个上下文记忆数据集, 用于模拟用户在环境中的行为。

- **指令数据集。** 我们考虑两种具有代表性的智能环境设置: 生活空间场景 (智能家居) 和工作空间场景 (智能办公室), 它们体现了多样化的用户查询需求。针对智能家居场景, 我们采用了先前工作中的智能家居指令数据集子集 [23], 包含**40条显式指令**和18条松散定义指令。为了更好地评估个性化性能, 我们进一步设计了15条额外的松散定义指令, 与 [23]中的指令结合后, 总计得到**28条松散定义指令**。针对智能办公场景, 我们从开放数据集 [19] 中收集了常见的工作场所需求, 并将其扩展构建成一个专为智能办公环境定制的指令数据集。两种场景的完整指令数据集均在附录A中提供。

- **上下文记忆数据集。** 我们使用对应的环境布局 (环境h3中的 [23]) 以确保公平比较。为了填充 MemAura中的上下文记忆, 我们构建了一个多用户共享事件日志数据集, 该数据集多样化地模拟了智能环境中的用户交互历史, 总共包含360条不同的24小时记录。我们的记忆上下文数据集的真实性基于一个真实世界数据集 [38]。我们利用大语言模型来丰富其多样性, 然后在VirtualHome物理引擎中回放生成的轨迹 [40] 以进行验证, 仅保留那些满足真实世界交互逻辑的记录, 从而确保我们数据集的真实性。具体的数据集采样和模拟流程记录在附录B中。

3.2 评估指标

我们提出了一套全面的指标来评估MemAura的性能。为清晰起见, 我们定义如下符号: 指令数据集记为 C_{com} , 上下文记忆数据集记为 C_{mem} 。令 D 为所有设备的集合。对于每个设备 $d \in D$, 令 F_d 表示与 d 关联的功能节点集合。对于每个用户 u 和指令 $com \in C_{\text{com}}$, 存在一个最小的设备集合 D_{goal} 和一个对应的功能集合 F_{goal} , 它们共同满足预期任务; 存在一个由大语言模型响应得出的设备集合 D_{res} 和功能集合 F_{res} ; 以及一个符合用户偏好的设备集合 D_{pre} 和功能集合 F_{dpre} 。基于这些定义, 我们的评估指标涵盖以下维度:

- **设备错误(DE)。** 当系统生成最终响应时, 如果它选择了错误的设备或未能包含必需的目标设备, 则视为发生了错误。

$$DE = \frac{1}{|C_{\text{com}}|} \sum_{com \in C_{\text{com}}} de(com, D_{\text{goal}}, D_{\text{res}}). \quad de(com, D_{\text{goal}}, D_{\text{res}}) = \begin{cases} 1, & \text{if } \exists d \in D_{\text{goal}} \text{ such that } d \notin D_{\text{res}} \\ 1, & \text{if } \exists d \in D_{\text{res}} \text{ such that } d \notin D_{\text{goal}} \\ 0, & \text{otherwise.} \end{cases}$$

- **功能错误(FE)。** 另一方面, 当未发生设备错误, 但系统在其最终响应中选择了错误的功能时, 则视为发生了功能错误。

$$FE = \frac{1}{|C_{\text{com}}|} \sum_{com \in C_{\text{com}}} fe(com, F_{\text{goal}}, F_{\text{res}}). \quad fe(com, F_{\text{goal}}, F_{\text{res}}) = \begin{cases} 0, & \text{if } de(com, D_{\text{goal}}, D_{\text{res}}) = 1 \\ 1, & \text{if } \exists d \in D_{\text{goal}} \text{ such that } F_{\text{res}}(d) \notin F_{\text{goal}}(d) \\ 0, & \text{otherwise.} \end{cases}$$

- **失败率(FailR)**。发生任一类型的错误都表示任务失败。

$$FailR = DE + FE.$$

- **准确率(Acc)**。当两种类型的错误均未发生时，任务被视为成功。

$$Acc = 1 - FailR$$

- **延迟**。我们通过记录从用户发出指令到大语言模型返回最终输出的时间，来衡量总响应延迟。
- **令牌消耗**。对于每个调用大语言模型的模块，我们还计算了总令牌消耗量，这使我们能够估算在实际部署场景中使用大语言模型完成任务所需的货币成本。
- **个性化率(PR)**。为了评估系统的响应是否足够有创意，我们认为当输出中包含符合用户偏好的设备选择或功能选择时，即发生了个性化。

$$PR = \frac{1}{|C_{com}|} \sum_{com \in C_{com}} pr(com). \quad pr(com) = pr(com) = \begin{cases} 1, & \text{if } (D_{res} \cap D_{pre}) \neq \emptyset \\ & \text{or } \exists d \in D_{goal}, F_{res}(d) \in F_{d_{pre}}, \\ 0, & \text{otherwise.} \end{cases}$$

3.3 基线方法与骨干模型

我们将我们的系统与三个基线进行比较：(1) 先前的工作 **Sasha** [23], (2) **Vanilla:Sasha** [23] 使用上下文记忆数据集中的原始事件日志进行增强，(3) **向量检索增强生成** [25]：其工作原理是将语料库存储为向量数据库中的密集向量嵌入。对于每个用户查询，它计算查询嵌入，通过最近邻搜索检索语义上最相似的块，并将检索到的内容作为上下文输入大语言模型。我们将MemAura中的上下文记忆管理模块替换为向量检索增强生成管道。我们使用嵌入模型 `sentence-transformers/all-MiniLM-L6-v2`，并在评估中根据经验确定其最优超参数：每条日志的最大长度为180，最大嵌入输入长度为256，批处理大小为32。对于每个意图单元，我们首先获取3K个候选，然后检索前 k 个结果，使用 $k=10$ 。

此外，为了检验不同大语言模型类型对系统性能的影响，我们使用以下模型评估所有基线和MemAura：**GPT-4o** [34]，一个由OpenAI提供并通过API访问的大型闭源模型；**phi-2** [32]，一个来自微软的轻量级模型，拥有27亿个参数（约3B），本地部署；以及**Qwen3-4B** [46]，一个近期发布的轻量级开源因果语言模型（具体为Qwen3-4B-Instruct-2507），拥有约40亿个参数，同样本地部署。

3.4 实现

在记忆图管理器中，我们使用 Neo4j 来构建和管理全局图，随后大语言模型可以发出查询语句，以快速定位其中的相关信息。在周期性模式预测器中，我们使用 `bert-base-cased` [7] 作为预训练模型，所有文本均使用标准分词器处理。考虑到 `bert-large-cased` 支持的最大输入长度为 512 个标记，仅保留最近的 512 个标记的行为序列，以确保符合模型的输入限制。在批次构建过程中，我们对序列应用伯努利掩码，掩码概率为 $p_{mlm} = 0.15$ 。优化设置如下：学习率 5×10^{-5} ，批次大小为 8，训练轮数为 3。所有实验均在 NVIDIA A6000 GPU 上进行。

接下来，我们将展示详细的评估结果，以回答研究问题1至研究问题5。

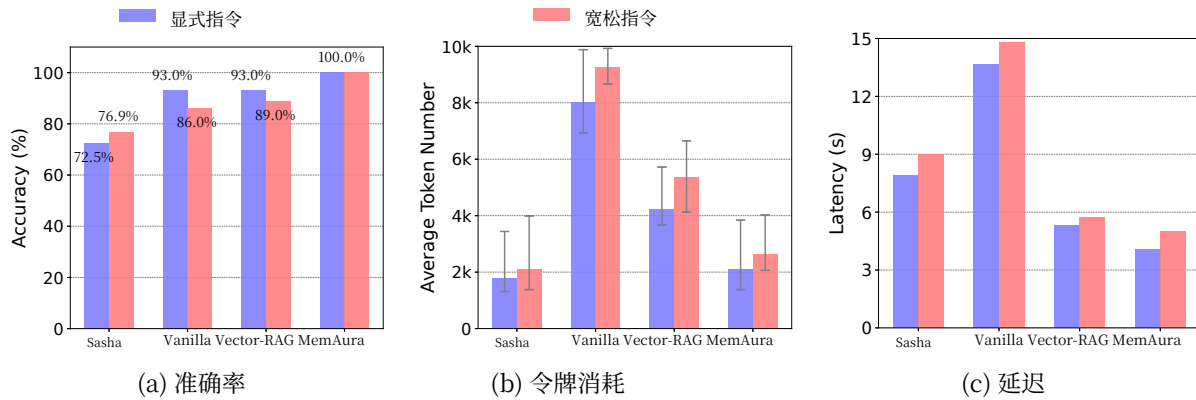


图 10. 使用 GPT-4o 作为骨干模型的整体性能与基线对比。(a) 展示了准确率的结果, (b) 呈现了令牌消耗的比较, (c) 报告了响应延迟。

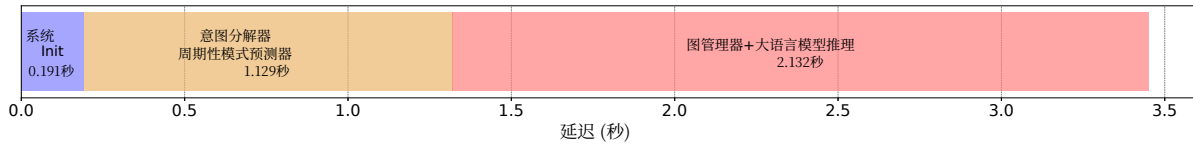


图 11. 使用 GPT-4o 作为骨干的 MemAura 延迟分解。我们并行运行无顺序依赖的组件。总体而言, 延迟可分为三个阶段以实现最佳实际性能。

3.5 使用大语言模型骨干的整体性能 (研究问题1)

我们首先进行了一项实验, 使用大型语言模型作为骨干, 来比较MemAura与基线方法在三个关键量化指标上的表现, 包括准确率、令牌消耗和延迟。本阶段所有实验均使用GPT-4o作为骨干模型。图10展示了结果。

在准确率方面, 结果显示Sasha (使用GPT-4o) 在显式指令上仅达到72.5%的准确率, 在宽松指令上为76.9%。当在原始基线中加入上下文记忆后, 与Sasha相比, 在两个指令数据集上的准确率分别提升了22.5%和15.4%。更值得注意的是, 使用GPT-4o的MemAura在所有实验中均实现了100%的准确率。这些结果表明, 融入上下文记忆能显著提升系统准确率。当我们将MemAura中基于图的记忆管理和检索替换为行业标准向量RAG方法时, 所得准确率与原始基线相当, 但仍分别比MemAura低7%和11%。这意味着我们基于图的记忆设计在检索简洁的上下文记忆以提升大语言模型推理准确率方面, 比向量RAG更有效。

在令牌消耗方面, 需要指出的是, 虽然原始基线方法提升了准确率, 但也显著增加了令牌消耗, 从大约2k令牌增至8k-9.5k令牌, 这是因为存储的大量记忆扩展了上下文窗口。与原始基线相比, 向量RAG减少了令牌使用量, 但仍比MemAura多消耗约1.9k-2.7k令牌。这是因为向量RAG检索到的日志不如基于图的检索那样简洁准确, 从而引入了冗余上下文。MemAura的结构化组织和选择性检索过程限制了传递给大语言模型的上下文数量。因此, 其令牌消耗维持在约3k, 仅比完全不考虑上下文记忆的Sasha多出约0.5k令牌。

在执行效率方面，MemAura 也优于各基线。Vanilla方法最慢，因为它包含了最多的上下文内容需要处理，导致响应时间达到12.2–14.6秒。向量RAG和MemAura 都达到了相近的执行效率水平，前者因需要搜索更多向量以找到相似条目而慢了约2秒。图11展示了MemAura推理延迟的详细分解。由于周期性模式预测器不依赖于分解后的意图单元，两者可以并行运行以减少端到端延迟。具体而言，意图分解器耗时1.129秒（32.71%），周期性模式预测器耗时约0.76秒。MemAura随后通过记忆图管理器查询偏好信息，并行调用大语言模型处理每个独立的意图单元，最后将它们的响应聚合为最终结果，此过程耗时2.132秒。总体而言，通过系统级优化和加速，MemAura实现了4.5–4.9秒的端到端延迟。

我们承认，MemAura目前尚无法提供近乎即时的反馈（例如，在1秒内）。然而，主要的瓶颈在于大语言模型的推理延迟。这也部分解释了为何Sasha会产生8-9秒的延迟，因为它需要多次调用GPT API。加速大语言模型本身很重要，但这超出了我们工作的范围。我们认为MemAura仍然具有价值，因为它仅带来最小的额外推理开销，同时显著提高了准确率，相比基线方法提升高达28%，这使得为了获得更好的服务而等待几秒钟是值得的。

3.6 使用轻量级大语言模型骨干的性能（研究问题2/研究问题3/研究问题4）

在本节中，我们将详细评估MemAura、Vanilla和向量RAG在融入上下文记忆后，在不同设置下的表现，并在两种场景下全面考察各项指标。结果如表2和表3所示。

观察1：MemAura 在使用轻量级骨干模型时保持高准确率。（研究问题2）如表2和表3所示，我们评估了Vanilla、向量RAG和MemAura在三种不同骨干模型下的表现。其中，phi-2（3B）和Qwen3-4B是轻量级模型，可以部署在家庭设备或本地服务器上。我们观察到，当使用GPT-4o作为骨干时，Vanilla方法能达到相当不错的准确率。然而，当骨干模型替换为Qwen3-4B或phi-2时，Vanilla的准确率急剧下降——分别下降了约43%和65%。这表明，如果没有大模型的大上下文窗口和强大的推理能力，仅提供原始历史日志不足以引导有效的推理。向量RAG在phi-2和Qwen3-4B上的准确率比Vanilla高出20–33%，但仍然逊色于MemAura。MemAura在不同模型间均保持了强劲的性能。以Qwen3-4B为骨干时，准确率保持在94–95%左右；即使使用phi-2，其准确率相比GPT-4o也仅下降了约12–23%。在使用轻量级大语言模型骨干时，MemAura与基线方法在宽松指令上的性能差距更大。例如，向量RAG的准确率比MemAura低约23–26%。一个可能的原因是，向量RAG检索到的上下文虽然比输入所有日志更简洁，但其准确性和精炼程度仍不及MemAura基于图的记忆管理所生成的上下文。由于轻量级模型（如Qwen3-4B和phi-2）相比大模型（如GPT-4o）推理能力有限，它们需要像MemAura所提供的这样更精炼、更具针对性的上下文来维持推理准确率。这表明，MemAura的有效性主要不依赖于骨干大语言模型的内在能力，而是源于MemAura所提供的结构化记忆表示和检索策略。

观察2：MemAura 很少产生设备选择错误，这与原始基线和向量RAG方法形成对比。（研究问题3）我们将错误分为设备错误和功能错误。如表2和表3所示，原始基线在这两类错误上出现的频率大致相当。向量RAG方法在智能家居和智能办公场景的宽松指令上表现出更多的设备错误。然而，MemAura几乎从不产生设备错误——也就是说，MemAura的响应不会错误地选择或遗漏所需设备。所有剩余的失败仅归因于功能选择的不准确。这一结果

表2. 在智能家居指令数据集上，MemAura与基线方法使用轻量级大语言模型的性能结果汇总。我们评估了多项指标，包括准确率（Acc）、设备错误（DE）、功能错误（FE）、延迟、令牌消耗和个性化率（PR）。

配置			总体			延迟			令牌消耗			个性化
方法	模型	指令准确率	DE	FE		最小值	最大值	平均值	最小值	最大值	Avg	PR
Vanilla	phi-2	显式	0.28	0.48	0.25	13.24	17.88	14.60	2008	2048	2021	0.10
		宽松	0.31	0.54	0.15	14.29	20.15	16.95	2019	2048	2051	0.15
	Qwen3-4B	显式	0.50	0.30	0.20	12.98	19.11	16.14	7980	9761	8799	0.18
		宽松	0.56	0.23	0.23	11.27	18.87	14.85	8024	10902	9570	0.16
	GPT-4o	显式	0.93	0.00	0.07	10.27	14.38	12.15	7425	7982	8142	0.10
		宽松	0.89	0.04	0.07	12.60	18.62	15.05	7902	9867	9363	0.19
向量检索增强生成	phi-2	显式	0.78	0.15	0.08	4.86	6.37	5.26	1740	2048	1806	0.25
		宽松	0.54	0.36	0.11	5.05	6.38	5.64	1939	2048	2002	0.15
	Qwen3-4B	显式	0.83	0.10	0.08	4.99	6.60	5.28	2797	3639	3244	0.20
		宽松	0.68	0.00	0.32	5.04	6.15	5.33	3152	5270	4165	0.18
	GPT-4o	显式	0.93	0.00	0.07	4.83	6.72	5.06	3931	5546	4328	0.30
		宽松	0.89	0.04	0.07	5.16	6.20	5.70	4126	6880	5285	0.25
MemAura	phi-2	显式	0.88	0.03	0.10	2.92	4.25	3.02	1678	2048	1890	0.38
		宽松	0.77	0.00	0.23	3.07	4.96	3.48	1897	2048	1992	0.31
	Qwen3-4B	显式	0.95	0.00	0.05	2.90	4.88	3.57	1024	1984	1453	0.25
		宽松	0.94	0.00	0.07	2.85	4.33	3.03	1628	3637	2234	0.35
	GPT-4o	显式	1.00	0.00	0.00	2.79	4.46	2.83	2004	3990	2004	0.38
		宽松	1.00	0.00	0.00	2.68	4.87	3.08	2595	4390	2987	0.43

得益于MemAura的结构化记忆管理：基于图的组织显式地编码了设备与其功能之间的关系，并为大语言模型提供了一种高效的针对性检索方法。虽然大语言模型能够读取JSON格式日志，但当设备和功能数量增加时，原始JSON数据变得冗繁，使得大语言模型难以可靠地提取和映射正确的执行单元。相比之下，MemAura的结构化图表表示实现了精确且高效的设备推理。

观察3：MemAura 在保持实际延迟的同时，有效降低了令牌消耗。复杂的任务和冗长的输入通常会导致基于大语言模型的系统响应时间和令牌使用量增加，尤其是在骨干模型轻量且能力有限的情况下。如果单个指令需要数十秒和数万个令牌，该系统在实际部署中显然是不切实际的。我们测量了MemAura、向量RAG方法和Vanilla在不同任务和模型设置下的延迟和令牌消耗。Phi-2的上下文窗口相对较小（约2048个令牌），而Qwen3-4B支持更大的上下文窗口（超过30K个令牌）。因此，如结果所示，使用phi-2的Vanilla几乎占满了整个上下文窗口，而使用Qwen3-4B时仍需要 ~8799–9570个令牌。可以合理推断，Vanilla输入超出了phi-2的上下文容量，因此被截断，这直接解释了准确率大幅下降的原因。即使使用Qwen3-4B，令牌消耗仍然很高，不适合实时使用。相比之下，MemAura的表现明显更好。尽管MemAura在使用phi-2时偶尔接近

表3. 在智能办公指令数据集上，MemAura与基线方法使用轻量级大语言模型的性能结果总结。我们评估了多项指标，包括准确率（Acc）、设备错误（DE）、功能错误（FE）、延迟、令牌消耗和个性化率（PR）。

配置			总体			延迟			令牌消耗			个性化
方法	模型	指令准确率	DE	FE		最小值	最大值	平均值	最小值	最大值	Avg	PR
Vanilla	phi-2	显式	0.30	0.35	0.35	12.37	16.08	14.50	2029	2048	2040	0.13
		宽松	0.29	0.58	0.13	16.33	20.90	18.39	1995	2048	2015	0.21
	Qwen3-4B	显式	0.50	0.30	0.20	14.20	18.92	16.14	7027	9782	8263	0.18
		宽松	0.54	0.21	0.25	13.60	15.16	14.58	7936	10926	9610	0.21
	GPT-4o	显式	0.93	0.00	0.07	12.04	15.33	13.66	6928	9882	8010	0.18
		宽松	0.86	0.11	0.04	12.98	16.42	14.82	8667	9926	9255	0.21
向量检索增强生成	phi-2	显式	0.78	0.18	0.05	4.79	6.32	5.14	1722	2048	1927	0.23
		宽松	0.57	0.21	0.21	4.90	6.48	5.46	1927	2048	2010	0.15
	Qwen3-4B	显式	0.78	0.18	0.05	5.17	6.53	5.02	2635	3895	3026	0.20
		宽松	0.71	0.00	0.29	5.12	6.15	5.30	3386	5113	4106	0.21
	GPT-4o	显式	0.93	0.00	0.07	4.85	6.68	5.30	3668	5723	4244	0.30
		宽松	0.89	0.07	0.04	5.14	6.80	5.74	4126	6652	5360	0.25
MemAura	phi-2	显式	0.93	0.05	0.03	3.09	5.03	3.99	1699	2048	1834	0.35
		宽松	0.82	0.00	0.18	3.30	5.74	4.38	1842	2048	1902	0.32
	Qwen3-4B	显式	0.95	0.00	0.05	3.29	5.21	4.22	1007	1877	1531	0.25
		宽松	0.93	0.00	0.07	3.30	5.11	3.31	1639	3620	2028	0.39
	GPT-4o	显式	1.00	0.00	0.00	3.09	4.82	3.07	1380	3842	2100	0.38
		宽松	1.00	0.00	0.00	3.10	5.35	4.00	2063	4026	2638	0.43

上下文限制，但其准确率仅略有下降，这表明丢失了最少的必要信息。在使用Qwen3-4B时，MemAura将令牌消耗保持在合理的1453–2234个令牌范围内。向量RAG使用的令牌数比Vanilla少得多，但仍比使用Qwen3-4B和GPT-4o的MemAura多。在智能家居数据集上，它分别比MemAura多使用约1.8k和2.3k个令牌；在智能办公室数据集上，它分别多使用约1.5k–2.1k和2.1k–2.7k个令牌。

观察4：MemAura 能生成更具个性化和创造性的响应。（研究问题4）个性化率反映了系统响应与用户偏好的一致程度。如表2和表3最后一列所示，引入上下文记忆后，原始基线可实现约10–19%的个性化率，而MemAura 则达到25–43%，相比原始基线提升了15–23%，且在两种场景下的指令数据集中均表现出稳定的性能。向量检索增强生成的个性化率比MemAura低约5–16%，在宽松指令上下降幅度更大。这是一个清晰且有意义的提升，表明MemAura的结构化记忆表征和有效的模式提取机制为大语言模型提供了更丰富、更准确的个性化信号。

需要澄清的是：尽管MemAura 在使用GPT-4o时达到了约43%的最佳平均个性化率，但这主要是因为指令数据集中有相当一部分内容（尤其是显式指令）不允许进行个性化。更多指令细节请参见附录A。

我们展示一个来自实验结果的示例，包括每个模块的输出以及由MemAura生成的最终响应：

指令：“我即将开始一个重要的在线会议。”

意图分解器：

[1] 为会议打开灯光 [2] 启用房间静音模式 **大语言模型选择**

的设备： [1] 卧室灯 [2] 台灯 [3] 卧室窗帘

周期性模式预测器：

用户可能操作设备：[1] 卧室窗帘 [2] 台灯 [3] 空气净化器 用户可能进行：[1]发言 [2] 打字 [3] 翻阅书籍
在过去相关时间，用户通常希望：[1] 提供安静环境 [2] 确保外出后安全 [3] 制作一杯卡布奇诺 记忆图管理器：设备与活动转移偏好：卧室灯 → 卧室窗帘 (66%)/ 台灯 (23%)，台灯 → 打字 (37%)/卧室窗帘 (30%)/空气净化器 (21%)，卧室窗帘 → 空气净化器 (78%)/ 空调 (12%)。相关设备及对应功能：[1] 卧室灯 <设置_亮度：220 (68%)；255 (24%)> [2] 台灯 <设置_亮度：200 (66%)，255 (10%)> [3]空气净化器 <设置_模式：节能模式 (90%)；睡眠模式 (5%)，设置_风速：低 (100%)> [4] 空调 <设置_温度：26°C (67%)；24°C (21%)，设置_模式：自动模式 (92%)；节能模式 (7%)，设置_定时器：8小时 (18%)；6.5小时 (14%)；2小时 (10%)>

高层级用户画像：

[1]...[7]当用户工作时，关闭卧室窗帘。 [8]...**MemAura：**明白。根据您的偏好，我将打开卧室灯并将其**亮度设置为220**。然后，我将打开台灯并将其**亮度设置为200**。之后，我将关闭卧室窗帘，打开空气净化器，并将其**模式设置为节能模式，风速调低**。注意到当前室内温度较高且卧室是封闭空间，我还将打开空调，将其**设置为自动模式，并将温度设定为26°C**。

在处理上述指令时，意图分解器首先将其分解为两个可执行的意图单元。与此同时，周期性模式预测器提供当前时间戳的预测用户模式。接下来，大语言模型并行处理每个意图单元。在识别出与指令相关的设备后，大语言模型通过记忆图管理器提供的接口查询其对应的偏好信息。高层级用户画像也作为指导性输入提供给大语言模型。最后，大语言模型基于这些信息进行推理，并提供个性化服务。在此示例中，绿色高亮部分表示可能有助于大语言模型进行个性化推理的信息，这也体现在其响应中。此示例表明，MemAura不仅满足了任务的功能性需求，还将用户特定的偏好纳入最终的行动计划中，从而产生更个性化、更符合用户长期习惯的响应。

3.7 消融研究 (研究问题5)

在本节中，我们对MemAura的各个模块进行消融研究，并基于代表性示例提供详细分析。本部分的所有实验均在智能家居指令数据集上进行，使用Qwen3-4B作为骨干模型。

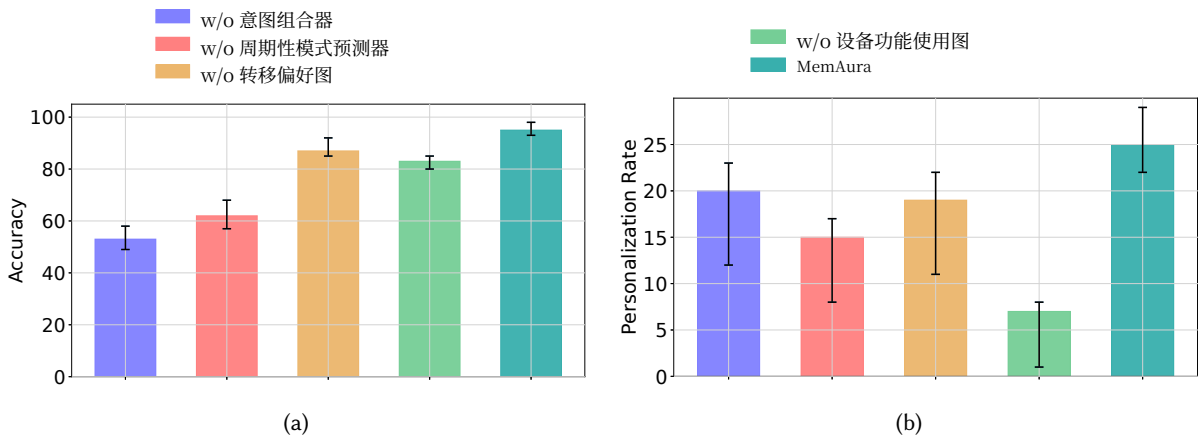


图12. 消融研究结果概览。(a)展示了移除各个组件时对MemAura准确率的影响，(b)展示了移除各个组件时对MemAura个性化率的影响。

3.7.1 意图分解器对准确率的影响。未经预处理的复杂任务会严重阻碍推理性能。如图12所示，当移除意图分解模块时，MemAura的准确率下降了近40%，而个性化率仅轻微程度下降（约5%）。这表明，当遇到复杂或模糊的指令时，未能将其分解为可执行的意图单元会显著损害性能，对于推理能力有限的模型尤其如此。

3.7.2 模式预测预测器对性能的影响。我们考察了如果要求模型直接从记忆中推断用户模式，而不经过程式提取过程，模型是否仍能完成任务。如图12所示，移除模式预测预测器导致任务成功率下降33%，更重要的是，个性化率下降10%。这表明，若不提供总结好的行为模式，像Qwen-3-4B这样的轻量级模型难以从记忆中提取个性化信息，也难以识别正确的执行逻辑。因此，个性化和准确性都显著降低。

3.7.3 设备与活动转移偏好图及设备功能使用图的影响。为了检验移除用户习惯信息的影响，我们分别消融了这两个子图并评估性能。如图12所示，当移除设备与活动转移偏好图时，MemAura的准确率降至87%，下降了约8%，个性化率降至19%，下降了6%。这表明转移偏好图对个性化的贡献更大，可能是因为它提供了行为模式信息，有助于模型推断用户接下来可能做什么。当移除设备功能使用图时，对个性化的影响更为显著：个性化率降至7%，下降了18%。这表明，若不了解用户通常偏好哪些功能和参数设置，系统将难以生成符合用户习惯的响应。对准确率的影响较小但仍可察觉，下降了约12%。总体而言，设备与活动转移偏好图和设备功能使用图对MemAura的性能都起着关键作用，并且它们对提升个性化率的影响尤为显著。

3.8 系统压力评估 (RQ6)

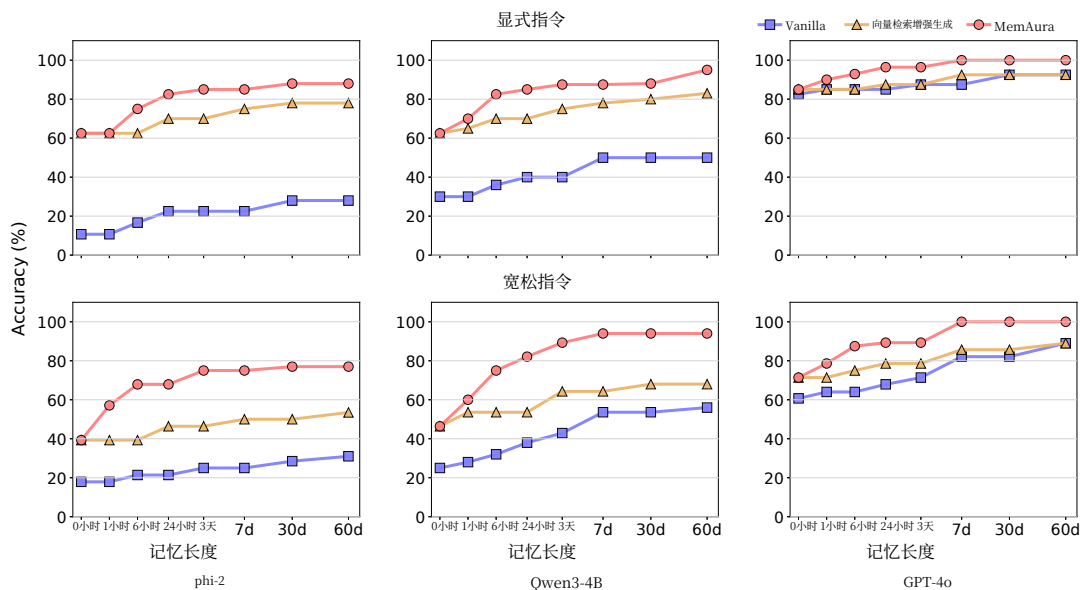


图 13. 压力测试 1 (冷启动): 不同记忆长度对准确率的影响

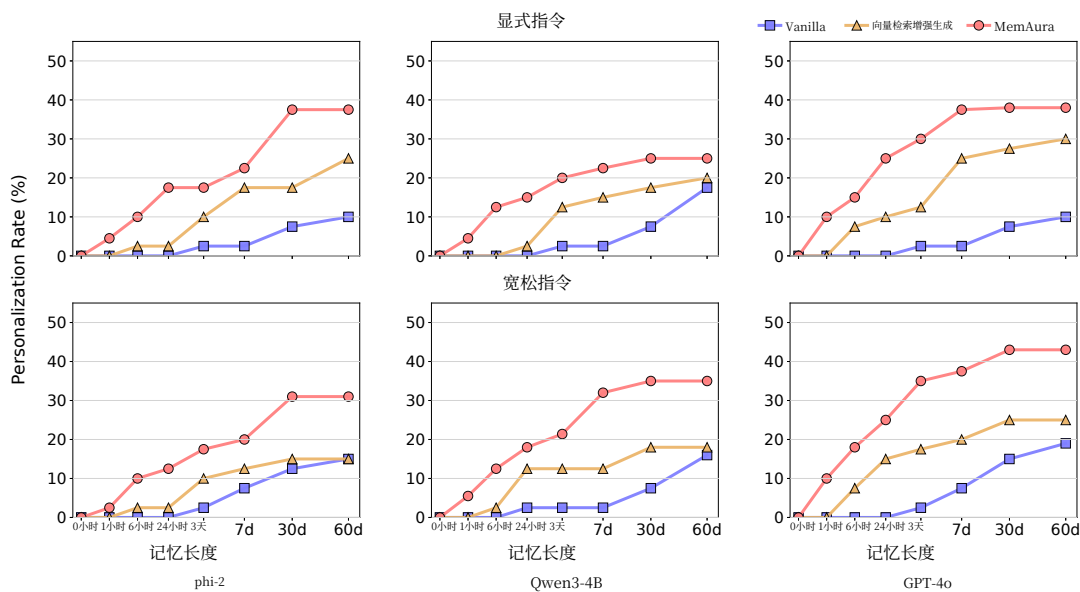


图14. 压力测试1 (冷启动): 不同记忆长度对个性化率的影响。

在本节中，我们设计了一系列压力测试，以模拟现实世界中的约束条件，并评估MemAura在面对这些情况时的鲁棒性。

3.8.1 冷启动场景：记忆长度对性能的影响。 我们首先模拟一个冷启动场景。MemAura组织记忆并支持准确高效的检索以实现个性化服务，但在初始阶段，当日志历史还很小时，我们想知道MemAura是否仍能带来有意义的收益。为了研究记忆长度的影响，我们应用一个滑动窗口，仅保留当前时间之前 $L \in \{0h, 1h, 6h, 24h, 3d, 7d, 30d, 60d\}$ 内的日志，其中h表示小时，d表示天。高层级用户画像和周期性模式预测器也仅使用同一窗口内的日志构建。当 $L = 0$ 时，我们将其视为冷启动，除了指令外不提供额外信息，而 $L = 60d$ 则使用完整的记忆。

在准确率方面，如图13所示，在冷启动条件下，向量RAG和MemAura表现相当。使用phi-2和Qwen3-4B时，它们在显式指令上的准确率约为62%，在宽松指令上约为40%。使用GPT-4o时，它们的冷启动准确率分别达到82.5%和71.4%。Vanilla的冷启动准确率要低得多，因为它遵循基于Sasha的流水线，仅包含设备选择、设备JSON输入和命令解析，导致与MemAura以及我们的向量RAG实现存在较大差距。虽然准确率通常随着 L 变短而下降，但MemAura在所有长度上始终优于向量RAG。使用约一周的日志 ($L=7d$)，MemAura实现的性能与使用60天日志相当，这在实际部署中是可行的。

在个性化率方面，有限内存的影响比其对准确率的影响更为严重，如图14所示，这并不令人意外。对于Vanilla，当 $L < 24h$ 时，个性化几乎无法实现。当 $L > 7d$ 时，增加 L 能显著提升Vanilla的个性化率，并且将窗口从 $30d$ 延长到 $60d$ 会带来更明显的增益。向量RAG方法在 $L = 6h$ 和 $L = 24h$ 之间显示出明显的转折点，此后个性化率显著上升。使用phi-2或Qwen3-4B时，其在 $L = 6h$ 的个性化率仅为0-2.5%左右，但一旦 $L > 6h$ ，便迅速增加；从6小时延长到24小时可带来7.5-10%的提升。使用GPT-4o时，转折点出现得更早，大约从 $L = 1h$ 开始。MemAura在 $L = 1h$ 处显示出转折点，其个性化率随着内存时长增加而稳步提升。在 $L = 24h$ 时，它达到了最终个性化率的大约一半，而在 $L = 30d$ 左右已接近 $L = 60d$ 水平。总体而言，MemAura展现了比基线更好的内存利用：即使只有1小时的内存，也能在不同骨干模型上实现一致的个性化增益。

3.8.2 设备中断/缺失日志的影响。 我们的第二次压力测试模拟设备中断。智能环境包含具有不同老化率的多样化设备，在实际应用中，某些设备可能离线，导致日志缺失。为了模拟设备中断和缺失日志，我们按如下方式注入故障：我们评估数据集集中的每个设备（共22个设备）都有概率 p_{out} 经历一次中断事件。如果发生中断，我们将移除该设备在持续时间为 $\delta \in \{8h, 24h, 7d, 15d, 30d\}$ 的连续中断块内产生的所有日志，该中断块在记忆窗口内均匀随机放置。其中， p_{out} 控制中断频率， δ 是中断持续时间。对于每个 p_{out} ，我们使用五个不同的随机种子重复随机中断注入，并报告平均值。

图15报告了MemAura在智能家居指令数据集上，使用Qwen3-4B模型时，面对设备中断的性能表现。在准确率方面，对于显式指令， $\delta \leq 24h$ 的中断对准确率影响甚微：仅当 $p_{out} > 10\%$ 时准确率才下降，即使 $p_{out} = 30\%$ 时仍保持在90%左右。对于 $\delta = 7d$ ，准确率保持在85%以上。在极端情况 $\delta = 15d$ 和 $\delta = 30d$ 下，当 $p_{out} \leq 10\%$ 时准确率高于85%，在 $p_{out} = 30\%$ 时仍保持78.5%。对宽松指令的影响稍大：在 $\delta = 8h$ 的情况下，准确率保持在89%以上，即使有30%的设备出现 $\delta = 30d$ 中断，准确率仍高于71%。在个性化率方面，对于显式指令，当 $\delta \leq 7d$ 时，个性化率仅在 $p_{out} > 10\%$ 时才开始下降；在 $p_{out} = 30\%$ 时，对于 $\delta = 8h, 24h, 7d$ ，个性化率分别保持在约17.5%、12.5%和10%。当 $\delta \geq 15d$ 时，中断的影响更大，但在 $p_{out} \leq 15\%$ 时，个性化率仍能保持约一半的原始水平。对于宽松指令，在 $\delta = 8h$ 的情况下，个性化率仅在 $p_{out} > 15\%$ 时才受到影响，并至少保持原始水平的67%；当 $\delta \geq 24h$ 时，其个性化率在

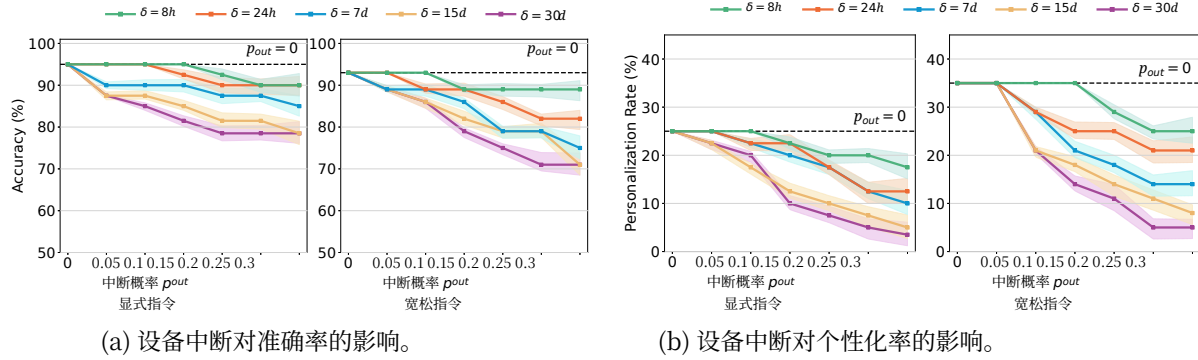


图 15. 压力测试2 (设备中断+缺失日志): p_{out} 表示设备发生中断的概率, δ 表示中断持续时间, 在此期间该设备无日志可用。

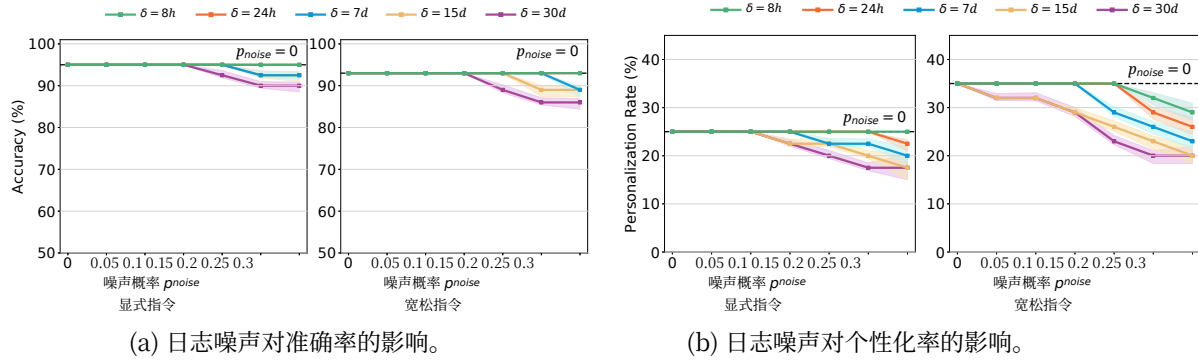


图 16. 压力测试3 (日志噪声): p_{noise} 表示噪声注入比例, δ 是注入噪声日志的时间窗口。

$p_{out} > 5\%$ 时开始下降。总体而言, 当 $p_{out} < 10\text{--}15\%$ 且 $\delta \leq 24h$ 时, MemAura的准确率和个性化率几乎不变。这表明MemAura对设备中断及由此产生的缺失日志保持了合理的鲁棒性水平。

3.8.3 日志噪声的影响。 我们的第三次压力测试模拟日志噪声。在实践中, 日志记录并非总是准确, 可能包含错误事件。为模拟此场景, 我们按如下方式注入错误日志事件: 每个现有日志以概率 p_{noise} 被附加一个额外的虚假条目。当被采样时, 我们根据数据集的实证事件分布抽取一个事件, 并将其插入当前日志与其原始后继日志之间。

图 16 展示了MemAura在智能家居指令数据集上, 面对日志噪声时, 基于 Qwen3-4B 的性能表现。就准确率而言, 注入的日志对显式指令和宽松指令的影响都很小。当 $\delta \leq 7d$ 时, 准确率几乎不变。仅在极端情况下 ($p_{noise} > 15\text{--}20\%$ 和 $\delta \geq 15d$), 准确率才会下降至 86–92.5%。就个性化率而言, 噪声对宽松指令的影响大于对显式指令的影响。对于显式指令, 当 $\delta < 7d$ 时, 个性化率没有明显下降, 但当 $\delta \geq 15d$ 和 $p_{noise} > 15\text{--}20\%$ 时, 会下降约 5–7.5%。对于宽松指令, 当 $p_{noise} \geq 20\%$ 时, 个性化率开始下降, 对于 $\delta = 8h$ 或 $24h$ (在 $p_{noise} = 30\%$ 时下降约 6–9%)。当 $\delta \geq 15d$ 时, 它持续下降至 $p_{noise} = 30\%$ 时的 20% 左右, 尽管这种情况

较为罕见。总体而言，在合理的设置下 ($\delta \leq 7d$ 和 $p_{\text{noise}} \leq 15\%$)，嘈杂的日志影响甚微，这表明 MemAura 具有相当的鲁棒性。

4 用户研究

除了定量性能评估外，我们还进行了一项用户研究，以评估MemAura的可用性。我们邀请了N = 8 名参与者对MemAura和基线的性能进行排序。随后，参与者可以自由指定场景和时间上下文，向 MemAura发出指令并评估其响应。此项用户研究旨在探究以下研究问题：

研究问题7：与基线相比，MemAura是否能提供更好的用户体验？

4.1 研究流程

4.1.1 参与者。我们招募了N=8 名参与者（年龄M = 23.6岁，标准差 = 3.4岁）。其人口统计信息如表4所示。

表4. 用户研究中参与者的基本人口学特征。

ID	Age	教育程度	智能环境熟悉度	熟悉的智能助理
P1	18-24岁	研究生	中等程度	Siri, ChatGPT语音
P2	25-34岁	学士	轻微程度	Siri
P3	35-44岁	研究生	轻微程度	Google助手
P4	25-34岁	学士	Very	Siri, ChatGPT语音
P5	45-54	高中或同等学历	中等程度	Siri, 小爱助手
P6	55-64	学士	轻微程度	Siri
P7	18-24岁	学士	Very	Siri, Gemini Home
P8	25-34岁	研究生	轻微程度	小爱助手

4.1.2 研究设计。为每位参与者提供了一台显示房间布局和设备分布的计算机，同时运行MemAura和基线系统以呈现响应结果。具体工作流程如下：

- (1) **人口统计问卷：**收集参与者的人口统计信息，特别关注其教育背景和对智能环境服务的熟悉程度。
- (2) **背景学习：**每位参与者都接受了15分钟的MemAura概述介绍，包括对上下文记忆数据集的详细解释，例如每位用户的特征和偏好。随后还演示了几个MemAura执行指令的示例。
- (3) **匿名排序与个性化评分任务：**在此任务中，在获知一个预定义的用户和指令后，参与者会看到由MemAura和基线模型生成的响应——他们不知道这些响应的来源，且顺序是随机的。参与者被要求根据整体性能将响应从最佳到最差进行排序，然后为每个响应的个性化程度评分，并提供简要的定性反馈。
- (4) **开放式问答任务：**在此阶段，参与者可以选择环境中的任何居民，并自由地向MemAura发出指令。之后，他们根据MemAura的响应来评估其性能。
- (5) **任务后问卷**
- (6) **半结构化访谈**

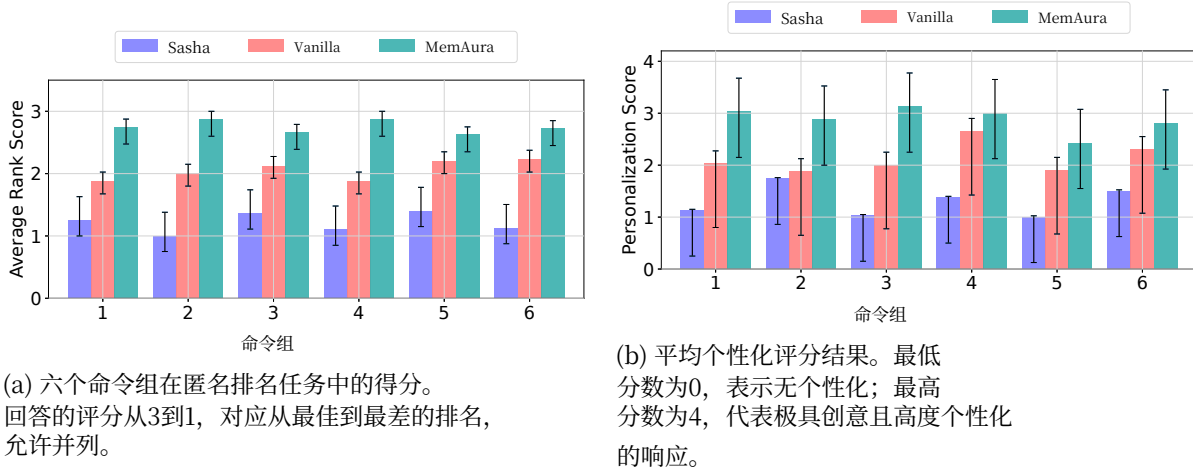


图 17. MemAura的匿名排序与个性化评分结果

在每项任务之间，参与者可以休息。研究结束后，参与者按每小时12美元的费率获得报酬。本研究已获得我们机构审查委员会（IRB）的批准。

4.2 匿名排序与个性化评分任务

4.2.1 匿名排序。 我们首先要求参与者完成匿名排名任务。在此任务中，指令根据六位不同的居民进行分组，每组包含从显式指令数据集中随机抽取的十条指令。MemAura和两个基线的响应以匿名和随机顺序呈现给参与者。参与者将响应从最佳到最差进行排序，分别对应3到1分。

如图17 (a)所示，MemAura 在排名任务中始终优于Sasha和Vanilla，平均得分在2.63到2.88之间，比Vanilla高出约0.7分。这些结果表明，从用户的主观视角来看，MemAura 的整体表现明显优于两个基线。

4.2.2 个性化评分。 接下来，参与者对每个响应的个性化程度进行评分。最低分0表示响应不包含任何个性化内容，最高分4表示响应提供了足够丰富且符合居民偏好的适当个性化。如图17 (b)所示，MemAura 的平均个性化得分约为2.63，显著高于基线。尽管与最高分仍有差距，但这是合理的——有些指令本身缺乏个性化的机会，或者用户的历史数据可能无法提供足够的证据来支持个性化响应。尽管如此，对比清楚地显示，Sasha和Vanilla的得分分别仅为1.26和2.0左右，这证明了MemAura在提供个性化服务方面的明显优势。

4.3 MemAura的用户体验研究

4.3.1 SUS问卷结果。 为收集用户对MemAura使用体验的评分，我们采用了系统可用性量表（SUS）[4] 问卷，该问卷被广泛用于评估系统可用性（附录C）。标准SUS问卷包含10个项目，采用5点李克特量表评分。我们按照 [20] 中描述的计分方法为每位参与者计算原始分数，即将1–5分的范围转换为0–4分。结果如图18 (a) 所示。随后我们还计算了总体SU分数以及可用性

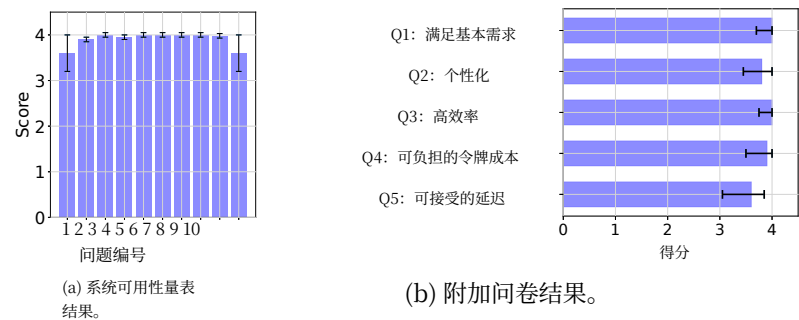


图 18. MemAura 的标准系统可用性量表及我们自行设计的问卷结果

子分数，分数范围为0至100。非专家组获得了98.00分 (± 3.00) 的SU分数，可用性子分数为98.75分 (± 1.875)。这一结果表明MemAura实用性很高，并提供了积极的用户体验。

4.3.2 自设计问卷结果。除了标准的SUS问卷外，我们还对MemAura用户体验中的特定问题感兴趣。因此，我们设计了一份包含以下问题的额外问卷：

- MemAura满足了用户指令的基本要求。
- MemAura提供了高质量的个性化服务。
- MemAura以高效率运行。
- MemAura的令牌消耗在可接受的范围内。
- MemAura的延迟在实际使用中是可接受的。

除最后一个问题外，上述所有问题均采用五点量表来评估参与者对陈述的同意程度，其中最低分表示“强烈不同意”，最高分表示“强烈同意”。采用五点量表是为了与SUS问卷保持一致。我们还将分值范围转换为0-4分。

我们自行设计的问卷结果如图18 (b)所示。大多数参与者认为，MemAura能有效满足用户指令的基本要求，且运行高效，令牌消耗保持在合理范围内。此外，参与者对MemAura提供的个性化质量和响应延迟给予了积极评价，表明该系统展现出强大的整体性能和实际可行性。

4.3.3 开放式问答任务。除了定量评估外，我们还邀请参与者参与一个开放式问答环节。参与者可以选择任意一位住户，在熟悉其用户画像和习惯后，自由地向MemAura发出指令。之后，我们鼓励他们评估MemAura的响应并提供反馈。

个性化质量体现在功能参数的细节之中。尽管每个人对个性化的定义可能不同，但真正的个性化往往体现在对偏好细节的精准处理上。当面对参与者模糊的指令时，MemAura 仍能推断用户偏好，并提供详细且符合情境的回应：

居民佐伊：一位家庭健身爱好者

P4: Pamela 的锻炼时间到了！我来准备好合适的空间。**MemAura:** 没问题！根据您的历史偏好，您通常从客厅中央开始锻炼，那里空调不会直吹。我会将空调设置为 26°C 并调至最低风速，打开客厅灯并将亮度设为 240，并启动音频播放器，播放您的爵士乐播放列表。我还会

拉上客厅窗帘以保护隐私。

P4 反馈: 这个回答真的很懂佐伊。我注意到她经常边听爵士乐边锻炼，对于一个女孩来说，锻炼时拉上窗帘是个非常贴心的细节。有时个性化不在于宏大的举动，正是这些细微之处让人感到温暖。

居民詹姆斯：一位咖啡爱好者

P6: 能给我来杯咖啡吗？**MemAura:** 当然可以。根据您的咖啡偏好，您通常在工作日上午享用一杯双份浓缩拿铁。我将启动咖啡机，将模式设置为拿铁，将研磨度调整到13，并选择双杯输出。由于天气宜人，且您偏好通风良好的环境，我也会打开厨房窗帘让新鲜空气进来，并逐渐调暗厨房灯光直至关闭。**P6 反馈:** 对于像我这样喜欢咖啡的人来说，这完全正确。它抓住了所有偏好的小细节。我们年纪渐长；有时连我妻子都会忘记如何调整设置，但系统还记得。

在上述两个例子中，参与者的指令并未包含显式细节，但MemAura成功地推断并应用了它们，这解释了积极的用户反馈。因此，准确提取并应用居民偏好的功能参数的能力，是实现个性化的一个关键因素。同时，对于老年用户而言，这种能力也能帮助他们记住日常设置细节，从而减少不便并提升可用性。

将近期历史纳入考虑。参与者设计了几个可能受近期事件影响的指令，以观察MemAura将如何响应：

居民哈里：一位宠物主人

两小时前的关键事件: 哈里给他的狗洗了澡。**P1:** 家里很杂乱，我们需要好好清洁一下。**MemAura:** 明白。根据您近期的历史记录，您刚刚给狗洗澡并吹干了毛发，因此彻底清洁浴室和走廊很重要。我将启动扫地机器人，使用您惯常的设置——模式设为先扫后拖，顺序为：卧室、客厅，然后是走廊，吸力设为最大。我还会打开浴室窗户和窗帘进行通风。30分钟后，机器人将再次清洁浴室，以防止毛发缠绕。**P1反馈:** 它记住了之前发生的事情，为我省去了一些麻烦。它知道哪些地方需要更多关注，并在水干后最后清洁浴室——非常贴心。

在此场景中，MemAura 生成了一项个性化服务，因为它知晓了之前的关键事件，从而有效避免了潜在的不便。另一位参与者为同一情境设计了不同的指令：

居民哈里：一位宠物主人

两小时前的关键事件：Harry 给他的狗洗了澡。
P2：我要去上班了。
MemAura：我将关闭热水器并重置的智能插座以将其功耗降至最低——只保留一盏客厅灯，亮度设为55。摄像头将设置为宠物模式。此外，由于记录显示你最近给狗洗过澡，我会将空调调整到27°C，并使用最低风扇风速，以防它着凉。**P2 反馈：**考虑得非常周到，利用历史记录来发现需要注意的事项。

从这些案例可以推断，先前的事件代表了一种隐含的偏好，因为它们直接影响后续的环境决策。因此，配备了能够高效检索历史上下文的记忆管理系统的MemAura，可以突出此类事件并将其纳入其响应中——这构成了个性化的基础。

在宽松指令中区分多个意图和任务人们通常更喜欢一次性表达一系列请求——毕竟，没人愿意一次只说一个指令然后等待回复，那样效率太低了。参与者还对MemAura能否在复杂或结构宽松的指令中有效区分多个意图，同时仍提供个性化服务感兴趣：

住户Jonathan

P8：我要进屋玩会儿游戏。得在我妈来之前把地方收拾干净，还要在她到了之后照顾她。
MemAura：好的。根据您的游戏偏好，我将打开卧室灯并将其亮度设置为180，关闭卧室窗帘，并关闭客厅灯。此外，我将立即启动扫地机器人，设置为快速模式并开启最大吸力。根据您的母亲来访时您通常使用的设置，我将设置一个触发器，以便当指纹ID 2解锁门时，客厅灯将以亮度255开启，并且音频播放器开始播放舒缓音乐歌单，音量为60。**P8 反馈：**这是一条相当长的任务链，但系统流畅地处理了所有事项，并为每个部分都添加了个性化设置。

由于MemAura中的意图分解器首先将一条指令分解为可执行的意图单元，因此即使是复杂的指令也仅仅是由多个并行执行的意图组成。这种设计不仅确保了MemAura处理复杂任务的能力，还使其能够区分用户对每个意图的偏好，从而实现更精准和个性化的服务。

5 相关工作

5.1 大语言模型的记忆管理

大语言模型（LLMs）最近在语言理解、生成和推理方面展现出卓越的能力，尤其是随着其上下文窗口的持续扩展，使其能够处理日益复杂的推理任务。为了增强大语言模型的知识并引导其行为，上下文组织、检索、管理和集成变得至关重要。早期的研究探索了各种上下文管理策略 [11, 35, 47, 59]，包括检索与生成机制，以及上下文处理、压缩和优化方法。这些后来演变为基于检索增强

生成 (RAG) 的系统, 以有效利用外部知识 [10, 14, 25, 45]。例如, FlashRAG [21] 引入了一个用于高效检索的模块化工具包, 而Self-RAG [3] 提出了一种自反思的检索策略。最近, 受人类认知启发, 研究人员开始将上下文视为一种记忆形式, 从而实现长期和短期信息的演化、更新和优化 [9, 27, 33, 51]。相关示例包括记忆库 [59], 它利用长期记忆来提升响应质量和个性化; 以及Sandbox [17], 它以交互方式管理长对话历史。然而, 当将记忆管理应用于现实环境以进行感知、交互和持续更新时, 这些方法面临着显著的局限性。大多数现有的基于记忆的系统主要设计用于聊天机器人式的对话场景, 而非像MemAura这样的智能环境。例如, THEANINE [33] 引入了一种基于时间线的记忆管理机制用于对话理解, 但仅支持自然语言文本记忆, 使其不适合处理物联网和环境设置中常见的数据。同样, A-MEM [54] 和MemoRAG [41] 专为对话或文档检索任务而设计, 但其存储的内容仍保持原始或非结构化格式, 需要大语言模型在运行时进行解释, 而无法像MemAura中所需的那样实现直接、结构化且即取即用的访问。

5.2 个性化大语言模型服务

个性化服务的概念可以追溯到推荐系统, 该系统基于历史记录及相关因素预测用户对某个项目的兴趣, 从而提供量身定制的产品或服务。传统的推荐系统通常依赖于自监督学习或其他数据驱动技术 [52, 60]。最近, 大语言模型驱动的推荐系统, 或更广泛地说, 基于大语言模型的个性化服务, 为该领域带来了新的可能性。一种直接的方法是将部分用户信息直接嵌入提示中, 以帮助大语言模型进行行为预测 [6, 36, 43, 48]。一些研究采用用户档案提示来改进推荐效果 [30, 44]。然而, 随着用户信息量的增加, 这种方法面临着可扩展性挑战, 例如超出上下文长度、处理不完整或不足的数据 [8, 39], 以及解决冷启动问题。因此, 大语言模型个性化本质上与上下文记忆管理紧密相关。例如, Li等人 [26] 构建了长期和短期对话记忆以及角色记忆, 以构建一个大语言模型驱动的个性化智能体; 而SeCom [37] 设计了一个片段级记忆库来提升检索效率。OS-1 [55] 使用眼镜设备收集数据, 这些数据随后被组织成实时记忆和历史上下文记忆, 并汇总成用户档案, 以帮助大语言模型在对话任务中提供更个性化的行为。受这些方法的启发, 我们在MemAura中设计了一个专门的记忆管理框架, 以实现面向智能环境定制的大语言模型驱动的个性化服务。

5.3 基于大语言模型的智能环境个性化服务

智能环境的定义性特征在于其普适感知与计算能力。联网智能设备、可穿戴传感器和嵌入式系统共同构成了一个智能生态系统, 持续与人类居民互动 [16, 53, 57]。近期研究利用大语言模型的推理能力来解读并作用于此类环境中产生的大量异构数据 [24, 29, 58]。智能环境的一个代表性应用是家庭医疗保健。例如, Kim等人 [22] 将可穿戴设备收集的生理信号输入大语言模型, 以提高诊断准确性。DrHouse [56] 使用来自家用设备的传感器信号作为输入来构建诊断决策树, 协助进行居家医疗咨询。这代表了另一种个性化服务的场景, 其中离散传感器数据必须转化为临床有用的情境信息, 这需要一个精心设计的记忆存储结构和信息提取机制。更常见的是, 智能环境体现在智能家居助手中。Sasha [23] 提出了一种更智能的基于大语言模型的智能助手的新范式, 探索如何

表5. 近期智能环境中LLM驱动服务的比较。'数据内容'和'数据格式'列分别描述了输入到大语言模型的数据类型和表示形式。'记忆时长'列表示存储数据的时间跨度——其中'情景式'指的是没有显式时间维度的非时序数据。

方法	数据内容	数据格式	记忆时长
GreenIFTTT [12]	设备信息	Json	情景式
传感器转文本 [5]	可穿戴设备数据	参数化形式	1-2小时
Sasha [23]	设备信息	Json	情景式
TaskSense [29]	传感器与设备数据	Json (→ 图)	6-12 小时
ContextAgent [58]	传感器与设备数据	Json	情景式
JARVIS [24]	HVAC传感器数据	Json (→ 检索增强生成)	1年 / 1分钟分辨率
MemAura	传感器与设备数据	Json (→ 图)	1440小时

大语言模型能够进行创造性推理并执行复杂任务。尽管它在处理模糊指令方面表现良好，但Sasha仅限于实时信息，缺乏任何时序上下文记忆，这导致其个性化响应能力较弱。在本工作中，我们也将Sasha作为详细评估的基线之一。Sensor2Text [5] 利用可穿戴传感器数据追踪日常活动，并支持自然语言交互。GreenIFTTT [12] 在类似IFTTT的框架中整合设备和传感器数据，并采用基于大语言模型的聊天机器人来帮助用户设计自动化例程。在商业层面，诸如Siri [2], Alexa [1]和Gemini for Home [13]等系统展示了该范式的实际实现。然而，如表5所总结，先前研究仍面临一个未解决的挑战：如何从连续的传感器和设备日志中提炼长期用户行为模式和偏好，以增强复杂任务中的个性化推理和服务质量。为应对这一挑战，我们提出MemAura，这是一个旨在高效管理上下文记忆并从原始物联网数据中提取行为模式和偏好的系统，以提升智能环境中大语言模型驱动的个性化水平。

6 讨论

所提出的MemAura为智能环境中的大语言模型个性化服务引入了一种结合上下文记忆的新方法。在本节中，我们将讨论MemAura在实际部署中的潜力、其隐私与安全考量以及当前工作的局限性。

6.1 在实际环境中部署MemAura

我们设计MemAura时考虑了实际部署需求，确保其多个组件和设计选择非常适合家庭环境。首先，骨干模型的选择提供了灵活性。如我们的评估所示，即使基于轻量级模型构建，MemAura也能保持稳定的性能和输出质量。因此，在实际部署中，用户可以选择（1）通过API调用高级大语言模型（例如GPT-4o、Gemini）以实现简化的工作流程和高准确率，或者（2）采用轻量级本地模型进行完全在设备上的操作。我们实验中测试的模型phi-2和Qwen-3-4B，都可以在家庭服务器或个人计算机上可行地部署，许多其他具备强大推理能力的紧凑模型也可以作为可行的替代方案。此外，基于BERT的周期性模式预测器可以在家庭服务器上本地训练。在1440小时物联网日志上训练的预测器大小约为4.5 GB，使得推理任务即使在个人笔记本电脑上也能流畅运行，这使其非常适合家庭使用。最后，数据存储需求适中：覆盖1440小时的多用户原始物联网日志仅占用几千字节，

存储开销极小。因此，MemAura对计算资源的需求相对较低，并且可以完全本地部署，为现实世界的智能环境提供了一个可行且保护隐私的解决方案。

6.2 基于LLM的MemAura的隐私与安全考量

由于智能环境不可避免地会从设备和传感器收集大量敏感数据，因此在任何实际部署之前都必须仔细解决隐私与安全问题。隐私风险是LLM驱动系统的一个关键关切点。最直接的缓解策略是在本地处理所有敏感信息，并在必要时仅将非敏感数据用于基于云的大语言模型。这种方法与我们之前关于本地部署的讨论高度契合，这自然消除了大多数隐私顾虑。具体而言，意图分解器模块可以安全地调用基于API的LLM，因为其输入完全由全局的、非私有的数据组成。然而，如果一个系统选择完全依赖先进的云端连接的大语言模型，则应包含相关机制，定期提醒用户在收集敏感数据时存在的潜在风险，并主动监控可能的数据泄露。总体而言，MemAura提供了部署灵活性，允许在隐私、性能和便利性之间取得平衡，确保敏感用户数据在智能环境中得到保护。

6.3 局限性与未来工作

我们的工作存在一些局限性。首先，我们系统中使用的物联网数据并非多模态的。许多多媒体物联网设备，例如摄像头和录音设备，会收集视频和音频模态的信息。尽管MemAura中当前的单模态记忆已经能够提供高质量的个性化服务，但融入多模态信息可能开启更具创造性和表现力的可能性。将这些新模态纳入记忆，引发了关于如何有效对齐、存储和协调异构数据的开放性问题，值得进一步探索。此外，MemAura的延迟仍有改进空间。尽管我们采用了意图单元的并行处理来减少响应时间，但实现近乎零延迟的服务仍然具有挑战性。未来的工作可以探索通过预训练或联合嵌入，使图结构记忆能够被模型直接解释，从而最大限度地减少知识蒸馏和输入处理所花费的时间。

7 结论

在这项工作中，我们提出了MemAura，一个用于智能环境的记忆管理系统，它通过三个关键模块组织设备和传感器记忆，提炼用户行为模式和偏好，并持续更新用户画像。实验结果表明，当由大型大语言模型驱动时，MemAura实现了100%的任务成功率和38%–43%的个性化率。即使使用轻量级大语言模型，MemAura仍能保持高达95%的准确率，平均个性化率为37.5%。相比之下，配备相同轻量级大语言模型的Vanilla仅实现28%–56%的准确率和最高17.5%的个性化率，而向量检索增强生成实现54–83%的准确率和最高25%的个性化率，这凸显了MemAura在模型能力受限情况下的鲁棒性和一致性。此外，MemAura在令牌使用和延迟方面表现出色，使其在实际部署中既高效又实用。

致谢

我们衷心感谢匿名编辑和审稿人提出的宝贵意见和建议。本研究得到了以下项目的支持：穗港（广州）联合资助项目（编号：SL2024A03J01192）、广州市科技计划项目（青年科技人才启航研究，编号：SL2024A04J01392）、广东省普通高校（青年创新人才，编号：2025KQNCX113）、广东省教育厅（创新团队，编号：2024KCXTD008），以及广东省泛在物联网通感算融合关键技术重点实验室（编号：2023B1212010007）。

参考文献

- [1] 亚马逊. 2025. alexa. <https://www.amazon.com/alexaplus> [2] 苹果公司. 2025. 苹果Siri. <https://www.apple.com/siri/> [3] Akari Asai, Zeqiu Wu, Yizhong Wang, Avirup Sil, and Hannaneh Hajishirzi. 2023. Self-rag: Self-reflective retrieval augmented generation. In NeurIPS 2023 workshop on instruction tuning and instruction following. [4] Aaron Bangor, Philip T Kortum, and James T Miller. 2008. An empirical evaluation of the system usability scale. Intl. Journal of Human-Computer Interaction 24, 6 (2008), 574–594. [5] Wenqiang Chen, Jiaxuan Cheng, Leyao Wang, Wei Zhao, and Wojciech Matusik. 2024. Sensor2text: Enabling natural language interactions for daily activity tracking using wearable sensors. Proceedings of the ACM on Interactive, Mobile, Wearable and Ubiquitous Technologies 8, 4 2024 1 26(), –. [6] Konstantina Christakopoulou, Alberto Lalama, Cj Adams, Iris Qu, Yifat Amir, Samer Chucuri, Pierce Vollucci, Fabio Soldo, Dina Bseiso, Sarah Scodel, Lucas Dixon, Ed H. Chi, and Minmin Chen. 2023. Large Language Models for User Interest Journeys. arXiv:2305.15498 [cs.CL] <https://arxiv.org/abs/2305.15498>. Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2018. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. CoRR abs/1810.04805 2018 [1810.04805](https://arxiv.org/abs/1810.04805) (). arXiv: . <http://arxiv.org/abs/1810.04805>. Horatiu Dumitru, Marek Gibiec, Negar Hariri, Jane Cleland-Huang, Bamshad Mobasher, Carlos Castro-Herrera, and Mehdi Mirakhorli. 2011. On-demand feature recommendations derived from mining public product descriptions. 2011 33rd International Conference on Software Engineering (ICSE) (2011 181 190 [2779250](https://doi.org/10.1145/1991446)), –. <https://api.semanticscholar.org/CorpusID:1991446> [9] Jizhan Fang, Xinle Deng, Haoming Xu, Ziyang Jiang, Yuqi Tang, Ziwen Xu, Shumin Deng, Yunzhi Yao, Mengru Wang, Shuofei Qiao, et al. 2025. LightMem: Lightweight and Efficient Memory-Augmented Generation. arXiv preprint arXiv:2510.18866 (2025). [10] Yunfan Gao, Yun Xiong, Meng Wang, and Haofen Wang. 2024. Modular rag: Transforming rag systems into lego-like reconfigurable frameworks. arXiv preprint arXiv:2407.21059 (2024). [11] Tao Ge, Jing Hu, Lei Wang, Xun Wang, Si-Qing Chen, and Furu Wei. 2023. In-context autoencoder for context compression in a large language model. arXiv preprint arXiv:2307.06945 (2023). [12] Mathyas Giudici, Luca Padalino, Giovanni Paolino, Ilaria Paratici, Alexandru Ionut Pascu, and Franca Garzotto. 2024. Designing home automation routines using an LLM-based chatbot. Designs 8, 3 (2024), 43. [13] 谷歌. 2025. 谷歌Home Gemini. <https://home.google.com/get-inspired/bring-home-the-magic-of-gemini/> [14] Zirui Guo, Lianghao Xia, Yanhua Yu, Tu Ao, and Chao Huang. 2024. Lightrag: Simple and fast retrieval-augmented generation. arXiv preprint arXiv:2410.05779 (2024). [15] Jiawei Hu, Hong Jia, Mahbub Hassan, Lina Yao, Brano Kusy, and Wen Hu. 2025. Lightllm: A Versatile Large Language Model for Predictive Light Sensing. In Proceedings of the 23rd ACM Conference on Embedded Networked Sensor Systems. 158–171. [16] Yongzhi Huang, Kaixin Chen, Yandao Huang, Lu Wang, and Kaishun Wu. 2023. A Portable and Convenient System for Unknown Liquid Identification With Smartphone Vibration. IEEE Transactions on Mobile Computing 22 (2023), 1894–1911. <https://api.semanticscholar.org/CorpusID:1991446> [17] Ziheng Huang, Sebastian Gutierrez, Hemanth Kamana, and Stephen MacNeil. 2023. Memory sandbox: Transparent and interactive memory management for conversational agents. In Adjunct Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology. 1–3. [18] ICATUS. 2023. Time-use activities classification Regional Workshop on Time Use Statistics: Methods and Uses. [19] IFTTT. 2024. IFTTT: Automation for Business and Home. <https://ifttt.com/>, Accessed:2024-09-06. [20] Brian Kenji Iwana and Seiichi Uchida. 2020. An Empirical Survey of Data Augmentation for Time Series Classification with Neural Networks. CoRR abs/2007.15951 2020 [2007.15951](https://arxiv.org/abs/2007.15951) (). arXiv: . <https://arxiv.org/abs/2007.15951> [21] Jiajie Jin, Yutao Zhu, Zhicheng Dou, Guanting Dong, Xinyu Yang, Chenghao Zhang, Tong Zhao, Zhao Yang, and Ji-Rong Wen. 2025. Flashrag: A modular toolkit for efficient retrieval-augmented generation research. In Companion Proceedings of the ACM on Web Conference 2025. 737–740. [22] Yu Han Kim, Xuhai Xu, Daniel McDuff, Cynthia Breazeal, and Hae Won Park. 2024. Health-LLM: Large Language Models for Health Prediction via Wearable Sensor Data. In ACM Conference on Health, Inference, and Learning. <https://api.semanticscholar.org/CorpusID:266998674> [23] Evan King, Haoxiang Yu, Sangsu Lee, and Christine Julien. 2024. Sasha: Creative Goal-oriented Reasoning in Smart Homes with Large Language Models. Proceedings of the ACM on Interactive, Mobile, Wearable and Ubiquitous Technologies 8, 1 (2024), 1–38. [24] Sungmin Lee, Minju Kang, Joonhee Lee, Seungyong Lee, Dongju Kim, Jingi Hong, Jun Shin, Pei Zhang, and JeongGil Ko. 2025. LLM-based Question-Answer Framework for Sensor-driven HVAC System Interaction. ArXiv abs/2507.04748 (2025). <https://api.semanticscholar.org/CorpusID:250704748> [25] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. 2020. Retrieval-augmented generation for knowledge-intensive nlp tasks. Advances in neural information

- 《ACM交互式、移动、可穿戴和普适技术汇刊》，第10卷第2期，文章55。发表日期：2026年6月。

[53] 徐立林、顾超杰、谭睿、何世波和陈积明。2023。MESEN: Exploit Multimodal Data to Design Unimodal Human Activity Recognition with Few Labels。 *Proceedings of the 21st ACM Conference on Embedded Networked Sensor Systems* (2023)。 <https://api.semanticscholar.org/CorpusID:268857170> 徐武江、梅凯、高航、谭俊涛、梁祖杰和张永锋。2025。

A-mem: Agentic memory for llm agents。 *arXiv preprint arXiv:2502.12110* (2025)。 [55] 徐振宇、徐海林、陆周阳、赵莹莹、朱锐、王玉江、董明志、常玉虎、吕琴、Robert P. Dick、杨帆、陆吞、顾宁和商丽。2024。Can Large Language Models Be Good Companions? An LLM-Based Eyewear System with Conversational Common Ground。 *Proc. ACM Interact. Mob. Wearable Ubiquitous Technol.* 8, 2, Article 87 (May 2024), 41 页。 doi:10.1145/3659600 [56] 杨步方、蒋思阳、徐立林、刘凯威、李海、邢国良、陈鸿凯、蒋晓凡和严振宇。2024。DrHouse: An LLM-empowered Diagnostic Reasoning System through Harnessing Outcomes from Sensor Data and Expert Knowledge。 *Proceedings of the ACM on Interactive, Mobile, Wearable and Ubiquitous Technologies* 8 2024 1 29 269929777 (), -。

[57] <https://api.semanticscholar.org/CorpusID:247131606> 杨步方、吴文轩、刘一天和刘红星。2022。A Novel Sleep Stage Contextual Refinement Algorithm Leveraging Conditional Random Fields。 *IEEE Transactions on Instrumentation and Measurement* 71 (2022), 1–13。 <https://api.semanticscholar.org/CorpusID:247131606> [58] 杨步方、徐立林、曾烈康、刘凯威、蒋思阳、陆文瑞、陈鸿凯、蒋晓凡、邢国良和严振宇。2025。ContextAgent: Context-Aware Proactive LLM Agents with Open-World Sensory Perceptions。 *ArXiv abs/2505.14668* (2025)。 <https://api.semanticscholar.org/CorpusID:278763319> 钟万军、郭良宏、高琪琪、叶赫和王彦霖。2024。

Memorybank: Enhancing large language models with long-term memory。 In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 38。 19724–19731。 [60] 周鑫、孙爱欣、刘勇、张杰和苗春燕。2023。SelfCF: A Simple Framework for Self-supervised Collaborative Filtering。 *ACM Transactions on Recommender Systems* 1, 2 (June 2023), 1–25。 doi:10.1145/3591469

指令数据集

A.1 智能家居场景指令数据集

指令编号	指令
显式指令	
1	让这里别那么寒冷
2	帮我凉快一下
3	让这里别那么闷热
4	外面寒冷时关掉空调
5	外面炎热时关掉暖气
6	让夜晚变得舒适
7	调暗一点
8	准备睡觉
9	帮我改善视线
10	日落后保持良好照明
11	太阳升起后使用自然光
12	我不在时保持灯光关闭
13	检查前门
14	锁好门
15	让客人进来
16	我离开时锁好门
17	有访客时通知我
18	白天确保家中安全
19	在我不需要时关闭恒温器
20	在我离开时节能

指令编号	指令
21	帮我节能
22	帮我降低电费账单
23	为聚会做准备
24	在这里营造舒适氛围
25	帮我放松休息
26	下雨时营造舒适氛围
27	天气不好时通知我
28	帮我改善睡眠
29	打扫卧室
30	停止清洁
31	整理房屋
32	日落之后停止清洁
33	有人时不清洁
34	我不在时进行清洁
35	给我一些隐私
36	让阳光进来
37	喝完咖啡
38	夜间保持湿润
39	我早上需要咖啡
40	天气好时让阳光进来
宽松指令	
1	我洗澡后太热，我想完成一些清洁工作
2	我想在餐厅桌上喝咖啡，看看外面
3	我看完比赛了，我要喝点咖啡，吃我的晚餐
4	人们要走了，我们让派对收尾吧
5	我要去睡觉了
6	我现在要放松一下
7	你能把阅读灯调成冷色调吗
8	我要出门了，请确保所有东西都已关闭
9	我去上班了
10	我朋友在厨房吐了
11	我不确定要咖啡还是茶。帮我选一个吧
12	帮我让我的女朋友开心起来
13	我的父母要来看我了
14	我饿了，我们准备点吃的吧
15	像往常一样观看点东西
16	祝贺我的生日
17	我的狗刚把地板弄脏了。我去给它洗个澡，你来处理剩下的
18	我马上要开始一个重要的线上会议了
19	这个周末我不休息。我会全天工作。
20	我会在客厅做些运动。
21	我想小睡一会儿。
22	外面的天气糟透了。别让我看到它。

指令编号	指令
23	我要出门，两天后才回来。家里的一切就交给你了
24	我到家了。现在准备去睡觉。去把其他地方通通风
25	门外是不是有人？确保我的安全
26	启动夜间模式
27	我会晚点回来，记得照顾好我的猫
28	让我们找点乐子吧

A.2 智能办公场景指令数据集

指令编号	指令
显式指令	
1	这里有点暗，你能帮我看得更清楚些吗
2	让这里稍微亮一点
3	你能把灯光调暗一点吗
4	营造舒适氛围
5	给房间一种更平静的氛围
6	让灯光更中性一些
7	请切换到冷色调灯光
8	帮我打开台灯
9	关掉我的台灯
10	请做一杯双份浓缩咖啡
11	感觉有点闷热，让空气流动起来
12	把这里弄凉快一点
13	稍微暖和一下
14	把它设到一个舒适的温度
15	打开空调
16	关闭空调
17	把风扇开到最大
18	将暖通空调系统设为自动模式
19	我的屏幕上有眩光，关闭百叶窗
20	把百叶窗打开一点
21	将百叶窗关到一半
22	让更多日光进来
23	我需要一些咖啡
24	空气感觉有灰尘，打开空气净化器
25	将净化器设置为静音模式
26	打开新风系统
27	关闭新风系统
28	将房间音量调高一点
29	将音量调低，太吵了
30	降低休息室的背景音乐音量
31	如果没人在观看电视，就把它关掉

指令编号	指令
32	我们需要浴室提供温水
33	关闭热水器以节能
34	浴室的水太烫了。请调整到更舒适的温度
35	在工作时间关闭洗衣机
36	请解锁房间门
37	锁上房间门
38	为访客打开主入口
39	清洁工作场所
40	清洁休息场所
宽松指令	
1	我感觉恶心又疲惫，你能帮我重置一下吗
2	这里太闷热了，但我不想让它变得寒冷
3	在这种氛围下我无法专注，让它变得好一些
4	让这个地方感觉舒适，但不要让人困倦
5	我想要更明亮，但我讨厌屏幕上的眩光
6	感觉嘈杂又混乱，你能让它更平静些吗
7	我觉得冰冷，但空气感觉干燥
8	我打算小睡一会儿，但不想搞乱今晚的睡眠模式
9	我很想喝咖啡，但之后可能会后悔
10	这地方太杂乱了，我需要它快速看起来体面些
11	我想进行清洁，但我不知道从哪里开始
12	我需要厨房在20分钟内准备好迎接客人
13	有什么东西闻起来不对劲，你能处理一下吗
14	我之前弄洒了东西，帮我处理一下
15	我需要放松休息，但我还有事情要完成
16	我累坏了，但明天会很紧张，帮我准备一下
17	今天这房间感觉不对劲……你能处理一下吗
18	我感到焦虑，你能帮我平静下来吗
19	我想把碗碟处理好，但现在不想听到噪音
20	我希望我的办公室在明天早上前准备好，找出最佳方案
21	我需要在這裡重新开始。
22	我需要进入“深度工作模式”一小时，然后把关掉。
23	我想要安静，但不是死寂。
24	让这里变得私密。
25	我马上要走了，确保我没有忘记任何重要的事情。
26	今晚我想感到安全和安定。
27	我要睡觉了，把环境调成睡眠模式，但别弄乱我的闹钟。
28	我马上要开个电话会议，把空间布置得看起来专业一些。

B 上下文记忆数据集的生成过程

大语言模型增强的真实世界行为数据集。我们基于真实世界数据集 HOMER [38]构建了我们的记忆数据集。原始数据集记录了五个家庭的日常作息，每个家庭有60条记录（总计300

条记录)。然而, 它仅包含22种高层级活动(例如刷牙或烹饪)的时间线, 缺乏详细的时间交互日志。我们首先填补了原始数据集中过长的空白间隔。我们遵循联合国统计司的《时间使用统计活动国际分类》[18], 其中包含约400个子活动。在过滤掉与HOMER不重叠的室内活动后, 我们指示一个大语言模型将它们插入原始数据集, 以保持空白段短于20分钟。大语言模型提示如下:

用于扩展原始数据集的提示。

你是一个智能家居助手。你需要填补每日活动时间线中的空白时段, 确保每个插入活动前后的语义连贯性。原始时间序列为: {timeline} 有些空白超过20分钟; 但不要填补在'离开_家'和'回到_家'之间的空白。以下是几个以CSV格式展示的填补示例: {example_csv_content} 在选择活动进行填补时, 请遵循以下规则: 1. 如果整个序列中不包含以下任何活动: {original_activity} →, 则优先从此列表中插入。2. 否则, 考虑从以下列表中插入: {additional_acticity}

接下来, 为了从有限的真实世界数据中获得更多样化的记录, 我们为每个家庭总结一个基础模式, 然后将其扩展为10个延伸模式, 以模拟更长期的记录。为了对更细粒度的行为进行建模, 我们使用少样本提示来引导大语言模型生成更真实、更详细的动作单元。大语言模型的提示如下所示:

用于模式生成和行为生成的提示。

你是一个智能家居助手。{raw_content} 是 {activity_name} 活动的基础模式。请基于此基础模式生成10个扩展模式。这些扩展模式可以在总持续时间、场景转换、家具和智能设备的选择、使用场景等方面有所不同。我希望你在扩展的同时, 保留原始模式的个人行为特征。这间房子里有一些智能设备: {device_list}

真实世界保真度保证。我们数据集的保真度植根于一个真实世界数据集 (HOMER), 而大语言模型极大地帮助使其更加全面和多样化。然而, 为了确保生成的序列在具身交互模型下是合理的, 而不仅仅是语言上合理, 我们在VirtualHome [40], (一个广泛使用的模拟引擎) 中验证它们。VirtualHome提供了一个执行引擎, 可以在三维场景中将日常活动作为原子动作运行, 同时维护显式的对象/智能体状态(例如, 开/关, 开/闭), 并在每一步强制执行三个约束: 对象可供性(目标支持该动作)、基于状态的前提条件和效果(所需状态必须成立并相应更新), 以及可达性(智能体必须能够导航到目标并与之进行物理交互)。因此, 无效的步骤将无法执行。我们将其用作可行性过滤器, 仅保留那些完整序列在模拟器规则下成功执行的日常活动。例如, 除非日常活动首先打开冰箱(并导航到可触及范围), 否则尝试将物品放入关闭的冰箱会被拒绝; 同样, 与对象支持的交互不匹配的动作(例如, “打开电视”)也是不允许的。为了在VirtualHome中执行我们数据集中的日常活动, 我们使用脚本将它们转换为所需格式的动作单元, 模拟每个日常活动, 并根据模拟分配时间戳以验证保真度。

多居民行为交织。上述步骤产生单用户模拟轨迹。为了研究多用户设置，我们进一步构建多用户家庭。我们首先计算每对用户在水时段内的DTW平均距离， $\delta = \text{avg}(\text{DTW}(A_i, B_i))$ ，其中较低的DTW距离表示较高的相似性。我们根据用户相似性来指导大语言模型的合并策略。对于DTW距离较低的用户对（即行为更相似），时间重叠活动被合并为一个联合执行事件。对于相似度较低的用户对，我们则将两个用户视为顺序执行活动。最后，我们将合并后的轨迹再次输入VirtualHome进行验证。然后，我们为每条记录分配一个从模拟中得出的时间戳，确保整个数据集的时间一致性和真实性。

C 问卷

1. 人口统计问题

您的年龄是?

- ☐ 18-24岁 ☐
25-34岁 ☐ 35-44
岁 ☐ 45-54岁 ☐
54+ 岁

您的最高教育程度是?

- ☐ 高中或以下 ☐ 学士学位 ☐ 研究生或专业学位 (硕士、
博士等)

您对智能环境的熟悉程度如何?

- ☐ 经常使用智能环境中的服务 ☐ 偶尔使用一些智能服
务 ☐ 对智能环境几乎或完全不熟悉。

(可多选) 您是否经常使用任何智能助手?

- ☐ 无 ☐ Siri ☐ Alexa ☐
Gemini Home ☐ 其他:
☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐

2. 系统可用性量表问卷

请根据您在实验中的实际体验，使用1到5分制回答以下问题，其中1分表示强烈不同意，5分表示强烈同意。

我认为我会愿意经常使用这个系统。

☐ 1 ☐ 2 ☐ 3 ☐ 4 ☐ 5

我觉得这个系统不必要地复杂。

☐ 1 ☐ 2 ☐ 3 ☐ 4 ☐ 5

我认为这个系统易于使用。

☐ 1 ☐ 2 ☐ 3 ☐ 4 ☐ 5

我认为我需要技术人员支持才能使用此系统。

☐ 1 ☐ 2 ☐ 3 ☐ 4 ☐ 5

我认为此系统中的各项功能整合得很好。

☐ 1 ☐ 2 ☐ 3 ☐ 4 ☐ 5

我认为此系统存在太多不一致之处。

☐ 1 ☐ 2 ☐ 3 ☐ 4 ☐ 5

我想大多数人应该都能很快学会使用这个系统。

☐ 1 ☐ 2 ☐ 3 ☐ 4 ☐ 5

我觉得这个系统使用起来非常繁琐。

☐ 1 ☐ 2 ☐ 3 ☐ 4 ☐ 5

我对使用该系统感到非常有信心。

☐ 1 ☐ 2 ☐ 3 ☐ 4 ☐ 5

在使用这个系统之前，我需要学习很多东西。

☐ 1 ☐ 2 ☐ 3 ☐ 4 ☐ 5

3. 附加问卷

请根据您在实验中的实际体验，使用1到5的评分标准回答以下问题，其中1代表强烈不同意，5代表强烈同意。

1. MemAura 满足了指令中描述的基本需求。

☐ 1 ☐ 2 ☐ 3 ☐ 4 ☐ 5

2. MemAura 满足了指令中描述的基本需求。

☐ 1 ☐ 2 ☐ 3 ☐ 4 ☐ 5

3. MemAura 非常高效。

☐ 1 ☐ 2 ☐ 3 ☐ 4 ☐ 5

☐ MemAura的令牌消耗是可承受的。

☐ 1 ☐ 2 ☐ 3 ☐ 4 ☐ 5

☐ MemAura的响应延迟是可接受的。

☐ 1 ☐ 2 ☐ 3 ☐ 4 ☐ 5

6. 请简要描述您使用MemAura的体验。